Optimization of Hull-Block Placement and Retrieval in a Congested Planar Storage System

Seung Woo Han[1], Yeong Chan Han[1], Yejin Yoo[1], Hee Chang Yoon[1] and Jong Hun Woo[1,2]

[1]Department of Naval Architecture and Ocean Engineering, Seoul National University, Seoul, Korea.
[2]Research Institute of Marine Systems Engineering, Seoul National University, Seoul, Korea

Seung Woo Han: nimp91@snu.ac.kr
Yeong Chan Han: hani010226@snu.ac.kr
Yejin Yoo: yooyj0314@snu.ac.kr
Hee Chang Yoon: gmlckd12@snu.ac.kr
Jong Hun Woo: j.woo@snu.ac.kr (corresponding author)

Acknowledgement

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025-00555741); the Technology Innovation Program (RS-2025-02372996) funded by the Ministry of Trade, Industry & Energy (MOTIE); the International Cooperative R&D program (Project No. 0022929) and the HRD Program for Industrial Innovation (RS-2025-02263945; RS-2023-KI002688) funded by MOTIE and the Korea Institute for Advancement of Technology (KIAT); and the project titled 'Fostering Talent in Advanced Ship Blue Tech (RS-2025-02221147)', funded by the Ministry of Oceans and Fisheries, Korea.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Data Availability Statement

Due to the nature of the research, due to commercial supporting data is not available

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used ChatGPT in order to improve the readability. After using this tool/service, the authors reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Optimization of Hull-Block Placement and Retrieval in a Congested Planar Storage System

ABSTRACT

This study develops retrieval path planning and placement decision algorithms to minimize hull-block relocations in congested planar storage areas within shipyards. In such environments, retrieval operations are frequently obstructed by surrounding hull blocks, leading to costly relocations. While previous studies often overlook realistic operational constraints, this study explicitly considers multi-directional transporters with weight capacity limits and buffer-aware retrieval planning, which increases planning complexity but improves practical applicability. For retrieval, a dynamic programming-based method is proposed to decompose each route into sub-paths and construct a value-guided retrieval path. For placement, a Simulated Greedy Placement (SGP) algorithm and a reinforcement learning framework integrated with Graph Neural Networks (GNNs) are introduced, offering complementary trade-offs between computational efficiency and solution quality. A Cardinal Graph Neural Network (CGNN) is further proposed to capture spatial directionality in grid-based layouts, enabling effective placement strategies beyond conventional GNN representations. Extensive experiments across different yard sizes and scheduling conditions demonstrate consistent improvements over baseline methods, underscoring the practical relevance of the proposed approach.

Keywords: Storage optimization, Shipbuilding industry, Dynamic programming, Deep reinforcement learning, Cardinal Graph Neural Network

Nomenclature

$\boldsymbol{S}$	Set of cells in storage yard
$\boldsymbol{S_E}$	Set of empty cells in storage yard
$\boldsymbol{S_B}$	Set of movable blocks in storage yard
$\boldsymbol{S_U}$	Set of immovable blocks in storage yard
$\boldsymbol{S_F}$	Set of available buffer space
$\boldsymbol{S_O}$	Set of cells adjacent to entrance
$\boldsymbol{S_{path}}$	Set of cells composing a retrieval path
$\boldsymbol{W_T}$	Set of weight capacity in each type of transporter
s_i	Cell i in storage system
d_i	Storage duration of block i
w_i	Weight of block i
g_k	Type of transporter k
m_l	Weight capacity of transporter type l
c_B	Cost of the path through the block
c_E	Cost of the path through the empty space
C_s	Scaling factor, a relative coefficient proportional to yard area, representing normalized yard size
$R(\boldsymbol{S}, \boldsymbol{S_{path}})$	Relocation count incurred in yard $\boldsymbol{S}$ along path $\boldsymbol{S_{path}}$
$P_{\varphi}(\boldsymbol{S}, s_T, W)$	Retrieval path returned by algorithm φ given yard state $\boldsymbol{S}$, target block position s_T, and transporter capacity W
$F(S, b_k, s_i)$	Transition function updating yard state $\boldsymbol{S}$ when block b_k is placed at cell s_i

1. Introduction

This study investigates retrieval path planning and placement in planar storage systems, aiming to minimize relocations of obstructing items. Conventional inventory storage systems, such as port container yards or multi-shuttle systems, typically allow three-dimensional item movement and provide preallocated access space for transporters(Li & Li, 2022). As a result, retrieval operations are relatively straightforward and offer a high degree of operational flexibility. In contrast, the system examined in this study involves planar storage of substantially larger and heavier items, where three-dimensional movement is infeasible and items are densely arranged to maximize spatial utilization(Park et al., 2007).

This structure imposes significant constraints on retrieval by limiting feasible paths and increasing operational complexity. Under planar transporter motion, any item obstructing a retrieval route must be relocated, and dense storage configurations make such interference frequent. Therefore, it is crucial to identify retrieval paths and placement strategies that minimize relocations. This reduces unnecessary movements, improves system efficiency, and enhances logistics flow while lowering operational costs. Representative examples include shipyard hull-block storage yards and cargo storage areas in automobile carriers.

This study focuses on hull-block storage areas in shipyards. Hull blocks (hereafter referred to as blocks) are large steel structures serving as subassembly units in ship construction, typically exceeding 10 meters in width and length and weighing more than 100 tons(Roh & Cha, 2011). During intermediate stages between production processes, these blocks are temporarily placed and stored in the yard. However, due to the complexity of shipyard operations, the storage duration of blocks varies significantly, and the limited space combined with the large size of the blocks often results in frequent relocation of obstructing blocks within the yard(Park & Seo, 2009).

The relocation of obstructing blocks during retrieval entails additional loading, unloading, and transportation, which consume substantial human and material resources. Furthermore, these operations require strict safety measures and considerable manpower, ultimately driving up logistics costs. Therefore, developing retrieval path planning algorithms that minimize block relocations, along with determining optimal placement strategies, is essential for improving logistics efficiency in shipyards.

In previous research, the problem has been defined as the Planar Storage Location Assignment Problem (PSLAP), introduced by Park and Seo (2010), Tao et al. (2013), and Hansen et al. (2020). Park and Seo (2009) and Tao et al. (2013) mathematically formulated the problem and proposed retrieval and placement algorithms based on metaheuristics and rule-based decision making, demonstrating improved performance. However, their approaches were limited in practical applicability and generalization, as transporter movement was constrained to linear directions.

More recently, Kim et al. (2020) and Zhong et al. (2025) proposed algorithms that account for multi-directional transporter movement using model-free deep reinforcement learning. In practical settings,

however, additional complexities emerge: the maximum load capacity of transporters limits the set of blocks that can be relocated, and the presence or absence of buffer space directly influences the number of required relocations. This study addresses these conditions by developing retrieval and placement algorithms for multi-directional transporters under increased complexity, explicitly incorporating weight constraints and buffer-space availability.

This study first proposes a dynamic programming (DP)–based retrieval path planning algorithm and then introduces two placement algorithms to minimize relocations: a simulation-based greedy placement (SGP) method and a reinforcement learning–based placement method integrated with a graph neural network (GNN).

For the retrieval path planning problem, the task of moving a target block is decomposed into subproblems of finding paths to intermediate positions, which are then extended step by step to the entrance to construct a complete retrieval path. The Minimal Blocking Path (MBP) algorithm estimates the value of each intermediate position based on obstructing blocks encountered, producing paths that pass through the fewest possible obstacles. In addition, to account for relocation variability arising from the availability of buffer space, the Buffer-aware Path planning (BAP) algorithm is introduced, which approximates the relocation count by applying region labeling to capture changes caused by accessible buffer space.

For placement, a SGP algorithm is introduced, where blocks are temporarily assigned to all candidate positions and the expected retrieval count is evaluated to select the best location. While the approach allows reliable estimation of retrieval variations that are otherwise difficult to predict, it suffers from an exponential growth in computational cost with increasing yard size. To address this limitation, a deep reinforcement learning–based placement algorithm is developed. In this approach, the yard is modeled as a grid-structured graph, and a GNN is employed to improve state representation. To capture directional characteristics of the spatial structure, the Cardinal Graph Neural Network (CGNN) is designed, incorporating directional information into the message-passing process. As shown by Oh et al. (2024), graph networks play a critical role in deep reinforcement learning for combinatorial optimization by encoding structural connectivity and patterns. Building on this, the proposed approach applies CGNNs to improve placement decisions in complex yard environments.

To evaluate the proposed methods, a simulation environment is constructed using shipyard block storage yard data under congested conditions, where more than 60% of the yard area is typically occupied. The results demonstrate that the buffer-aware retrieval algorithm and the CGNN-based improving overall efficiency. While the SGP approach achieved the best performance in smaller scale settings, the CGNN-based algorithm showed superior effectiveness in larger yards by enabling dynamic decision making under complex conditions.

The remainder of this paper is organized as follows. Section 2 reviews related studies on logistics

optimization and prior research on PSLAP. Section 3 defines the problem in detail. Sections 4 and 5 present the proposed retrieval path planning and placement algorithms, respectively. Section 6 evaluates the algorithms across diverse problem settings and discusses the underlying causes of the results. Finally, Section 7 concludes with a summary of the study, highlighting key contributions and limitations.

2. Literature review

2.1. General studies on placement and retrieval

Research on optimizing placement and retrieval in general storage systems has primarily focused on automated warehouse systems, particularly multi-shuttle systems(Cinar & Zeeshan, 2020; Gagliardi et al., 2012). This stream of research is commonly referred to as the Storage Location Assignment Problem (SLAP), which has been addressed in representative studies by Yang et al. (2015), Silva et al. (2020) and He et al. (2024). Yang et al. (2015) proposed an efficient search algorithm based on Variable Neighborhood Search (VNS) to optimize storage location assignment as well as placement and retrieval scheduling in multi--shuttle systems with reusable shared spaces. Silva et al. (2020) integrated the storage location assignment problem with the order picking problem into a single optimization model and developed a solution method based on an improved General Variable Neighborhood Search (GVNS). He et al. (2024) considered both inbound and outbound operations simultaneously and performed cost optimization using an enhanced genetic algorithm (GA) under a physical modeling framework.

These studies contributed by proposing effective algorithms for placement and retrieval in logistics systems that allow three-dimensional stacking. However, these studies differ fundamentally from the focus of this study, which addresses the constraints of planar storage systems for large and heavy items. Specifically, the present work concentrates on the detailed processes of placement and retrieval in environments where retrieval is hindered by interference from other blocks and requires relocations.

2.2. Studies on hull-block logistics

Research on shipyard hull-block logistics has been conducted extensively due to its importance, with studies largely divided into two areas: transportation and storage. Most prior work has concentrated on optimizing transportation processes. Woo et al. (2010) and Nam et al. (2022) utilized actual shipyard data to perform digital twin simulations and proposed layout optimization approaches to improve block logistics. Roh and Cha (2011), Kim and Joo (2012) and Han et al. (2025) focused on transporter scheduling to optimize block transportation. Roh and Cha (2011) and Kim and Joo (2012) introduced metaheuristic-based solution search algorithms employing GA and ACO, while Han et al. (2025) proposed a dynamic scheduling algorithm based on graph reinforcement learning.

From the perspective of block storage, representative studies include Varghese and Yoon (2005) and Jeong et al. (2018), both of which addressed space allocation in block yards with scheduling considerations. Varghese and Yoon (2005) applied the Bottom-Left Fill (BLF) strategy for space allocation and then optimized scheduling using a genetic algorithm. Jeong et al. (2018) mathematically formulated the problem and proposed an optimization method based on a greedy algorithm. While these studies share similarities with the present work in considering space allocation with respect to block storage durations, they did not incorporate the detailed processes of inbound and retrieval operations and thus overlooked the issue of obstructing block relocations. The research most directly related to the present study, focusing on retrieval path planning and placement in planar storage yards, is discussed in the following section.

2.3. Studies on Planar Storage Systems

Representative studies on the PSLAP include Park and Seo (2009), Park and Seo (2010), Tao et al. (2013), Hansen et al. (2020), Kim et al. (2020), and Zhong et al. (2025). A summary of these works is provided in

Research addressing multi-directional transporter movement includes Hansen et al. (2020), Kim et al. (2020), and Zhong et al. (2025). Hansen et al. (2020) studied cargo placement in automobile carriers and proposed effective placement algorithms using a Greedy Randomized Adaptive Search Procedure (GRASP) and an Adaptive Large Neighborhood Search (ALNS). However, their approach only focused on placement decisions and approximated retrieval evaluation with simple rules rather than performing simulation-based analysis. Kim et al. (2020) developed deep reinforcement learning (DRL) based retrieval and placement algorithms for multi-directional transporters in block storage systems.

Table 1. Existing studies can be grouped into two categories: studies on linear transporter movement and studies on multi-directional movement. Multi-directional movement greatly expands the solution space and increases problem complexity, and research has therefore progressed from linear to multi-directional transporters. Studies that focused on placement and retrieval algorithms with linear transporter movement include Park and Seo (2009), Park and Seo (2010) and Tao et al. (2013). Park and Seo (2009) formulated PSLAP as a mathematical programming model and proposed an optimization method based on a GA. Park and Seo (2010) introduced a dynamic PSLAP priority rule (PR) that outperformed conventional rules and GA-based algorithms. Tao et al. (2013) presented a priority rule based on degrees of freedom for block placement and retrieval and combined it with a Tabu Search (TS) strategy to optimize the sequence of operations while minimizing relocations.

Research addressing multi-directional transporter movement includes Hansen et al. (2020), Kim et al. (2020), and Zhong et al. (2025). Hansen et al. (2020) studied cargo placement in automobile carriers and proposed effective placement algorithms using a Greedy Randomized Adaptive Search Procedure (GRASP) and an Adaptive Large Neighborhood Search (ALNS). However, their approach only focused on placement decisions and approximated retrieval evaluation with simple rules rather than performing simulation-based analysis. Kim et al. (2020) developed deep reinforcement learning (DRL) based retrieval and placement algorithms for multi-directional transporters in block storage systems.

Table 1. Previous studies of planar storage location assignment

Author	Placement	Retrieval	Transport movement	Weight constraint	Contribution
Park and Seo (2009)	GA	PR	Linear	X	· First propose mathematical modeling and GA for PSLAP
Park and Seo (2010)	PR, GA	PR	Linear	X	· Efficient heuristic rule · Measure schedule change impact
Tao et al. (2013)	PR, TS	PR	Linear	X	· Freedom based heuristic placement · Task sequence optimization by TS
Hansen et al. (2020)	GRASP, ALNS	-	Multi-direction	X	· Placement with GRASP and ALNS · Rule based placement evaluation
Kim et al. (2020)	DRL	DRL (Success/Fail)	Multi-direction	X	· The first multi-direction retrieval · Effective placement with deep RL
Zhong et al. (2025)	PR, DRL	PR, DRL	Multi-direction	X	· Propose zone-based placement for heterogenous size blocks · Rule guided effective RL for retrieval
This study	GNN, DRL	DP	Multi-direction	O	· Complex problem with weight-restricted relocations and buffer aware retrieval · DP based optimized retrieval path · SGP and GNN integrated DRL for effective placement strategy.

This study was the first to introduce a retrieval algorithm for PSLAP with multi-directional movement and showed that reinforcement learning–based placement strategies outperformed conventional heuristics. However, the retrieval algorithm focused primarily on feasibility rather than optimization and often failed under congested yard conditions with high storage density. Zhong et al. (2025) proposed a zone-based layout strategy that allocated blocks according to size and developed a DQN-based algorithm guided by priority rules. While their work contributed by considering block size heterogeneity, the reliance on rule-guided learning restricted the flexibility of the algorithm.

Although these studies established a variety of effective strategies for retrieval and placement with multi-directional transporters, they overlooked several critical factors present in practical environments. In reality, transporter weight capacity constrains which blocks can be relocated, resulting in a distinction between movable and immovable blocks. The presence of immovable blocks significantly increases the complexity of retrieval path planning, limiting the applicability of previous strategies. Similarly, placement strategies must account not only for storage duration but also for block weight to effectively support retrieval operations. To address these limitations, the present study proposes novel retrieval and placement algorithms that explicitly incorporate weight constraints and buffer-space availability, enabling effective decision making even in congested planar yards.

The key contributions of this study are as follows:

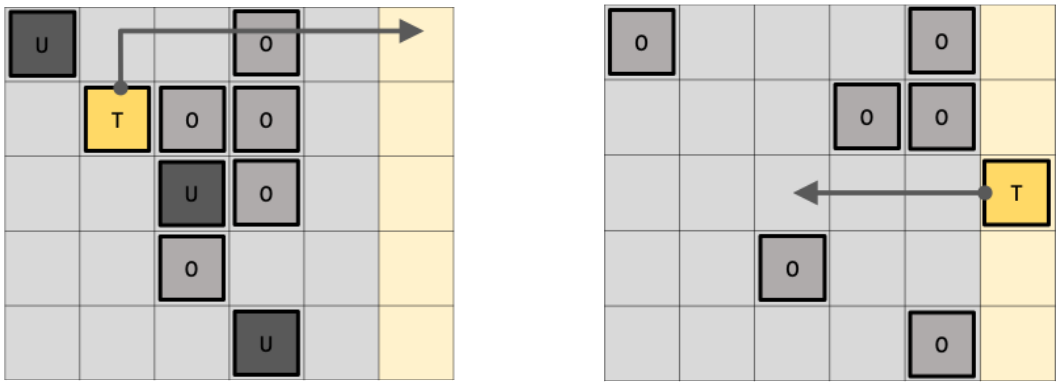
1. Formulation of a congested planar shipyard storage problem with weight-induced non relocatable blocks and buffer-aware retrieval, and development of a dynamic programming–

based retrieval path planning algorithm tailored to these constraints.

2. Proposal of a SGP and a GNN-integrated reinforcement learning method for placement decisions, providing a practical tradeoff between solution quality and computational efficiency.
3. Design of a CGNN for grid-based spatial graphs that explicitly encodes directional structure, improving the performance of the integrated reinforcement learning approach beyond conventional GNN representations.

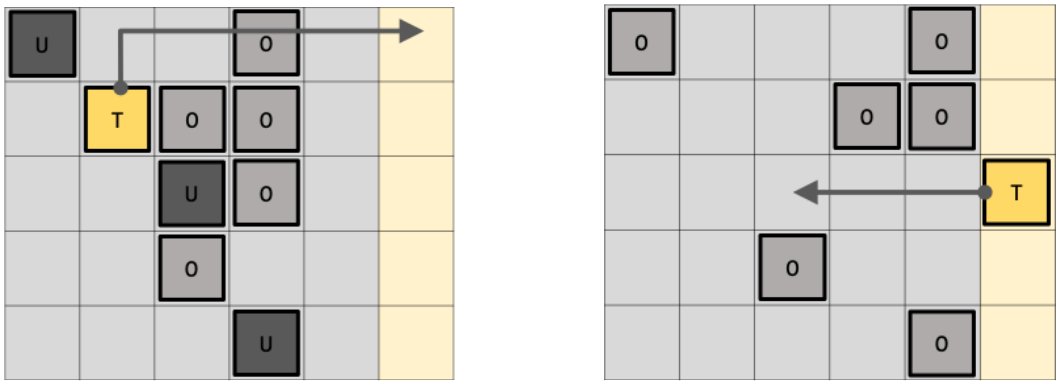
3. Problem definition

The problem addressed in this study is defined in two parts: retrieval path planning and placement decision.



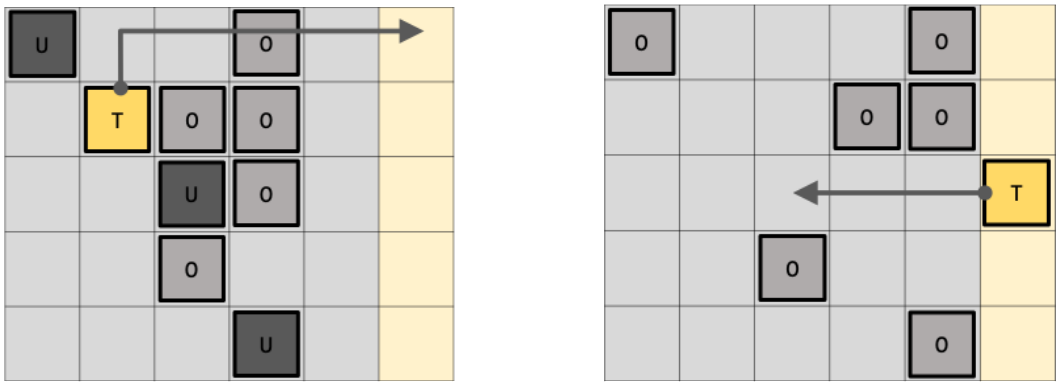
Retrieval path planning (a) Placement decision (b)

Figure 1 illustrates both problems, with the left (a) showing retrieval path planning and the right (b) showing placement. The problem environment is a planar storage system represented as a two-dimensional grid with a single entrance connected to a road. The yellow cell on the right marks the entrance through which transporters enter and the exit through which blocks are retrieved. Transporters can move up, down, left, and right, and are subject to a single load constraint. If obstructing blocks are present along the path, the transporter cannot pass until they are relocated. Depending on block weight, some obstructing blocks can be moved while others cannot. In



Retrieval path planning (a) Placement decision (b)

Figure 1, gray blocks labeled O represent movable obstacles, dark blocks labeled U represent immovable ones, and T denotes the target block for retrieval or placement. Detailed descriptions of each problem are provided in the following sections.



Retrieval path planning (a) Placement decision (b)

Figure 1. Diagram of retrieval and placement problems.

3.1. Retrieval path planning problem

The retrieval path planning problem is defined as finding a path from the target block to the entrance with the minimum number of relocations of obstructing blocks, given the current state of the yard and the designated block for retrieval. When the planar storage is represented as an M×N grid, the overall space $\boldsymbol{S}$ is defined as shown in Eq. (1). The entrance area $\boldsymbol{S_o}$, which is adjacent to the road, is defined as in Eq. (2). Let $\boldsymbol{S_B}$ denote the set of cells occupied by movable blocks, $\boldsymbol{S_U}$ the set of cells occupied by immovable blocks, and $\boldsymbol{S_E}$ the set of empty cells. All of these are subsets of the overall space $\boldsymbol{S}$.

$$\boldsymbol{S} = \{s \mid s = (i,j)\ \ 0 \le i < M,\ 0 \le j < N\} \tag{1}$$

$$\boldsymbol{S_o} = \{s \mid s = (j,0)\ 0 \le j < N\} \tag{2}$$

The retrieval path planning problem is defined as finding a retrieval path $\boldsymbol{S_{path}}$ from the target block to the entrance that minimizes the number of relocations of obstructing blocks, given the environmental conditions of the system $\boldsymbol{S_B},\ \boldsymbol{S_U},\ \boldsymbol{S_E}$ and the position of the target block $s_T \in \boldsymbol{S_B}$. This problem can be formulated as Eq. (4).

$$\min_{\boldsymbol{S_{path}}=(s_0,\ldots,s_k)} \boldsymbol{R}\left(\boldsymbol{S}, \boldsymbol{S_{path}}\right) \tag{4}$$

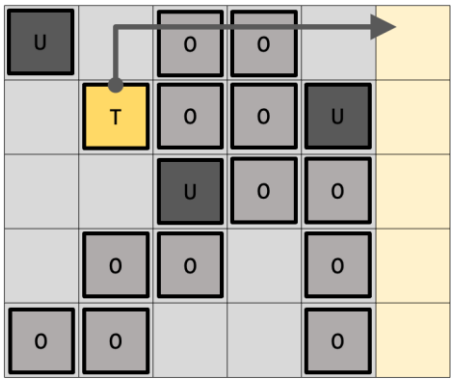
$$\text{s.t. } s_0 \in \boldsymbol{S_o}$$

$$s_i \in \boldsymbol{S_B} \cup \boldsymbol{S_E}, \forall i = 1, \dots, k$$

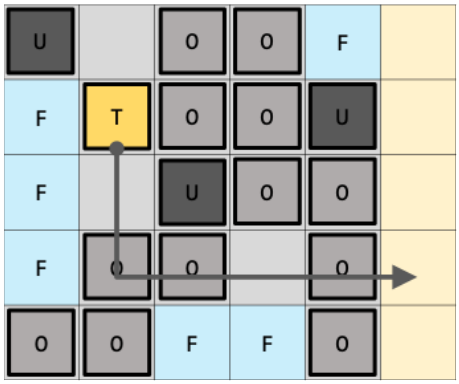
$$s_k = s_T$$

$$\|s_{i+1} - s_i\|_1 = 1, \forall i = 1, \dots, k$$

Here, $\boldsymbol{R(S, S_{path})}$ denotes the number of relocations required in yard $\boldsymbol{S}$ along the retrieval path $\boldsymbol{S_{path}}$, and its value depends on the availability of buffer space. Buffer space refers to accessible empty cells along the path that are not part of the retrieval route itself. When buffer space is available, obstructing blocks can be relocated directly within the yard. Otherwise, they must be temporarily removed and re-stored.



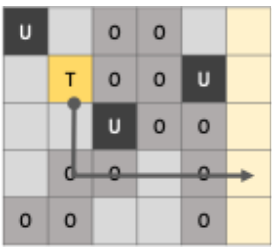
Retrieval path without buffer space (a)



Retrieval path with buffer space (b)

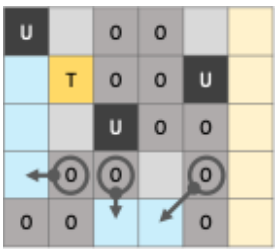
Figure 2 illustrates two cases. In the left (a), the black arrow indicates a retrieval path without buffer space, where two obstructing blocks must be temporarily moved outside the yard before the target block can be retrieved and then restored afterward. This process results in four relocations, which is twice the number of obstructing blocks. In contrast, the right (b) shows a retrieval path with available buffer space, where obstructing blocks can be relocated directly to accessible free cells within the yard, leading to only three relocations, equal to the number of obstructing blocks. Cells marked with F in blue denote accessible buffer space. Detailed retrieval processes for each case are presented in (a) Retrieval case 1

Step 1



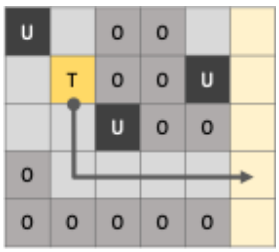
Target block path with buffer

Step 2



Relocate the obstacle blocks directly into buffer

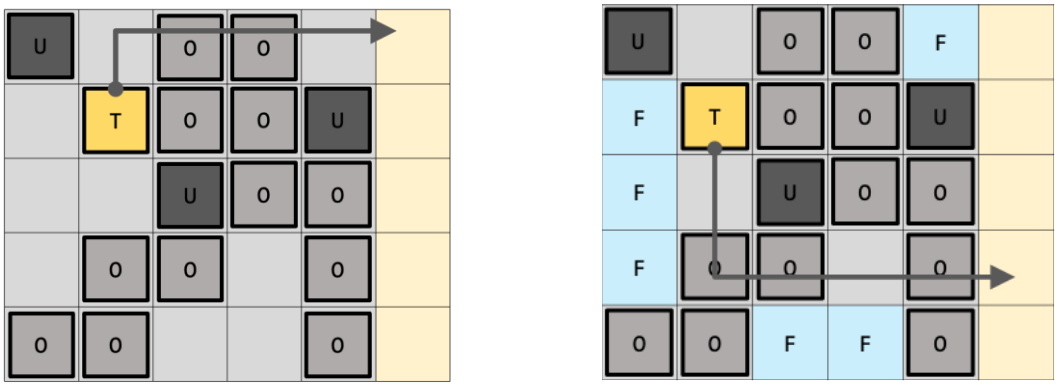
Step 3



Retrieve target block

(b) Retrieval case 2

Figure 3.

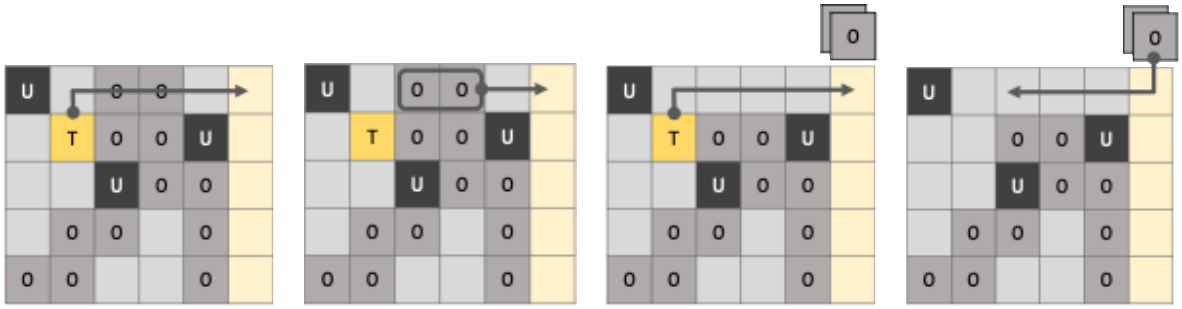


Retrieval path without buffer space (a) Retrieval path with buffer space (b)

Figure 2. Comparison of retrieval paths with and without buffer space

Once the retrieval path is determined, the transporter enters from the entrance and sequentially relocates obstructing blocks to secure the path. The target block is then retrieved along the secured path, and any temporarily relocated blocks are re-stored. The retrieval procedure is described in Algorithm 1, with $\boldsymbol{S_F}$ representing accessible buffer spaces and $\boldsymbol{S_P}$ representing feasible re-storage position.

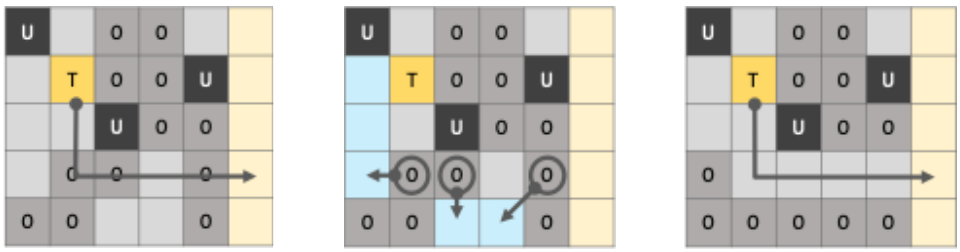
Step 1 Step 2 Step 3 Step 4



Target block path without buffer | Temporarily retrieve the obstacle blocks | Retrieve target block | Re-store the obstacle blocks

(a) Retrieval case 1

Step 1 Step 2 Step 3



Target block path with buffer | Relocate the obstacle blocks directly into buffer | Retrieve target block

(b) Retrieval case 2

Figure 3. Detailed retrieval processes with and without buffer space

Algorithm 1 Retrieval Process

Input: S, S_{path}
Output: $S', R(S, S_{path})$
Initialize variables:
$Count \leftarrow 0$
Initialize queue Q as empty
for $s \in S_{path} \cap S_B$ **do**
  Find $S_F \subset (S_E \setminus S_{path})$ such that each cell is adjacent to s
  if $S_F \neq \varnothing$ **then**
    Relocate block from s to a selected cell $s' \in S_F$
    $Count \leftarrow Count + 1$
  else
    Temporarily retrieve block to the outside
    Push block onto Q
    $Count \leftarrow Count + 1$
  end if
end for
Retrieve target block at s_T
while Q is not empty **do**
  Pop temporarily retrieved block from Q
  Find $S_P \subset S_E$ such that each cell is adjacent to S_o
  Select a placement cell $s \in S_P$
  Place block into s
  $Count \leftarrow Count + 1$
end while
Update stockyard state: $S' \leftarrow S$
Set $R(S, S_{path}) \leftarrow Count$
Return $S', R(S, S_{path})$

Algorithm 1. Retrieval process with block relocation

3.2. Placement decision problem

The placement decision problem is defined as determining the optimal storage location whenever placement occurs, either for incoming or relocated blocks. Its objective function, consistent with that of retrieval, is to minimize the number of relocations of obstructing blocks. Under a simulation framework where placement and retrieval are repeated over multiple days, the aim is to minimize the cumulative number of relocations. The overall simulation process is illustrated in Figure 4.

Within the yard, an initial set of blocks is randomly placed, and additional blocks are generated and stored daily. Each block is defined by a storage duration and a weight. Placement must be performed in empty cells, and because blocks cannot pass through one another during transportation, a feasible path to the placement position must be ensured. Details of path feasibility are provided in Section 5.1. Once the storage duration of a block expires, it is retrieved from the yard. Within each simulation cycle, all placement operations are completed before retrieval operations are executed (Kim et al., 2020).

For both placement and retrieval, transporters are allocated. Each transporter has a maximum load capacity determined by its type, and in the simulation a transporter with sufficient capacity for the block to be placed or retrieved is randomly assigned. As described in Section 3.1, this load capacity distinguishes between movable and immovable blocks. Accordingly, the placement decision problem is formulated as minimizing the cumulative number of relocations occurring during the simulation process.

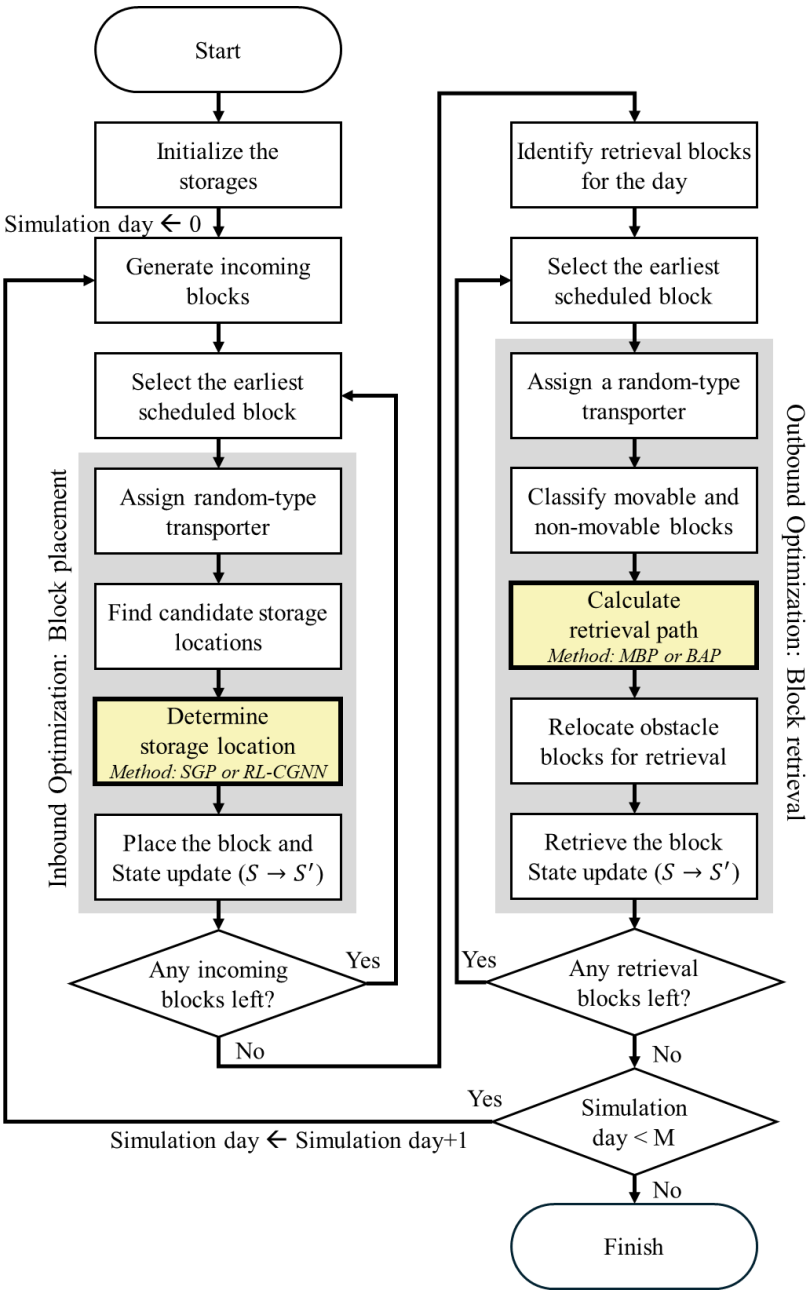


Figure 4. Flowchart of block placement and retrieval simulation

4. Retrieval algorithms

Retrieval path planning was based on a dynamic programming approach. The problem was decomposed into subproblems of moving the target block from its location to each cell. Cell values

were then propagated backward from the target using breadth-first search, representing accumulated relocation costs. A retrieval path to the target was subsequently derived by tracing from the entrance through cells with the highest values. The dependence of relocation counts on buffer space availability makes exact computation challenging. To address this issue, two approximate algorithms were developed: the Minimal Blocking Path (MBP) and the Buffer-Aware Path (BAP).

4.1. Minimal Blocking Path Algorithm

The Minimal Blocking Path algorithm seeks a retrieval path that passes through the minimum number of obstructing blocks without explicitly considering buffer space. Starting from the target location, cell values are propagated backward, where passing through an obstructing block is assigned a higher cost than passing through an empty cell. The formulation is given in Eqs. (5) ~ (7), where $V(s)$ denotes the value of cell s and $C(s, s')$ represents the cost incurred when moving from s to s′. Although path length minimization is not included in the objective function of this study, a small cost is also assigned to traversing empty cells so that the algorithm produces a unique path and prefers shorter routes among those that involve the same set of obstructing blocks. Algorithm 2 presents the update procedure of $V(s)$, and Algorithm 3 describes the process of deriving the retrieval path by following the highest-value cells from the entrance.

$$V(s) = \begin{cases} V_{init},\ if\ s \in \boldsymbol{s_T} \\ -\infty,\ \ if\ s \in \boldsymbol{s_U} \\ max_{s' \in A(s)}[C(s, s') + V(s')],\ \ otherwise \end{cases} \tag{5}$$

$$C(s, s') = \begin{cases} -c_B\ \ (c_B > 0),\ if\ s \in \boldsymbol{s_O} \\ -c_E\ (c_E > 0),\ if\ s \in \boldsymbol{s_E} \end{cases} \tag{6}$$

$$A(s) = \{s' \mid s' = s + a,\ a \in \{(1,\ 0), (-1,\ 0), (0,\ 1), (0, -1)\}, s' \in \boldsymbol{S}\} \tag{7}$$

Algorithm 2 Value propagation I

Input: $\boldsymbol{S}, s_T$
Output: $\boldsymbol{V}$
Initialize variables:
set $V(s_T) \leftarrow V_{init}$
Initialize queue $\boldsymbol{Q}$ as empty
Adjacent directions, $A \leftarrow \{(0,1),(0,-1),(1,0),(-1,0)\}$

push s_T into $\boldsymbol{Q}$
while $\boldsymbol{Q}$ is not empty **do**
  Pop s from Q
  for each a $\in$ A **do**
    $s' \leftarrow s + a$
    if $s' \in (\boldsymbol{S_E} \cup \boldsymbol{S_B})$ and $V(s') \leq V(s) + C(s,s')$ **then**
      $V(s') \leftarrow V(s) + C(s,s')$
      Push s' into $\boldsymbol{Q}$
    end if
  end for
end while
return $\boldsymbol{V}$

Algorithm 2. Value propagation for minimal blocking path algorithm

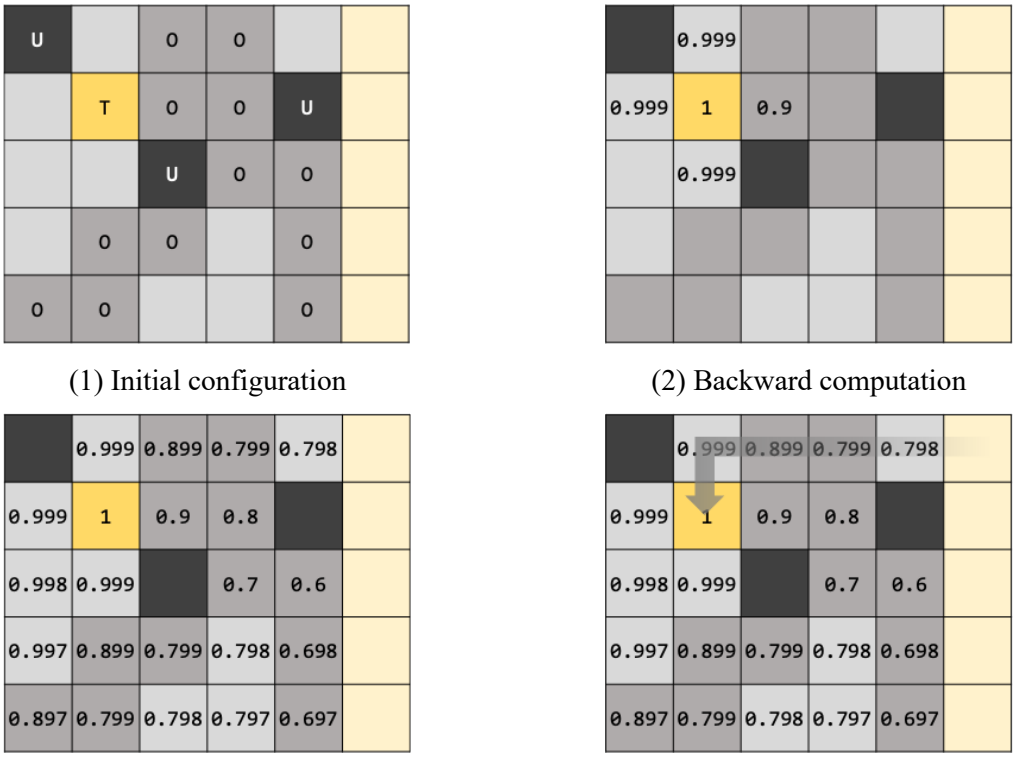
Algorithm 3 Value-Guided Path Extraction

Input: $\boldsymbol{S}, \boldsymbol{V}, s_T$
Output: $\boldsymbol{S_{path}}$
Initialize variables:
Adjacent movement, $A \leftarrow \{(0,1),(0,-1),(1,0),(-1,0)\}$

$s_0 \leftarrow \arg\max_{s \in \boldsymbol{S_{entrance}}} V(s)$
$s \leftarrow s_0$
Initialize $\boldsymbol{S_{path}} \leftarrow [s]$
while $s \neq s_T$ **do**
  $s' \leftarrow \arg\max_{s' \in \{s+a \mid a \in A, s+a \in \boldsymbol{S}\}} V(s')$
  Append s' to $\boldsymbol{S_{path}}$
  $s \leftarrow s'$
end while
return $\boldsymbol{S_{path}}$

Algorithm 3. Value-guided path extraction algorithm

As an illustrative example, when the initial value at the target position V_{init} is set to 1, the cost of passing through an obstructing block c_B is set to 0.1, and the cost of passing through an empty cell c_E is set to 0.001, the retrieval path of the target block is derived as shown in Figure 5.



(1) Initial configuration (2) Backward computation

(3) Value propagation (4) Trace highest path

Figure 5. Application of the minimal blocking path algorithm.

4.2. Buffer-aware path planning

The Buffer-Aware Path algorithm is a dynamic programming–based path planning method that incorporates the impact of buffer space availability on relocation counts. Similar to the MBP, cell values are propagated backward from the target location, with accessible buffer space accumulated during this process. When passing through an obstructing block, the assigned cost is doubled in the absence of buffer space compared with the case where buffer space exists. This adjustment reflects the situation described in Section 3.1, where relocating an obstructing block requires two operations if direct relocation within the yard is infeasible, necessitating temporary removal and re-storage.

Since buffer space is defined as accessible empty cells outside the retrieval path, both the total number of empty cells and the cumulative path length to the current cell must be tracked to properly evaluate buffer availability. In this context, available buffer space is quantified as the number of accessible empty cells outside the retrieval path, adjusted by subtracting the obstructing blocks encountered along the path.

The BAP algorithm begins with a region labeling procedure to identify connected empty spaces within the grid. Starting from the upper-right corner, connected empty cells are sequentially searched and labeled, and each connected set of empty cells is denoted as G_i, where i represents the region index. The formulation of the BAP is given in Eqs. (8) ~ (13), where $T(s)$ denotes the path length of the best

route to cell s, and $E(s)$ denotes the cumulative number of empty cells accessible along the best path to s. As shown in Eq. (11), the path length increases by one regardless of the type of cell traversed, while Eq. (14) shows that the number of accessible empty cells increases whenever a new region is encountered. To avoid double-counting, $L(s)$ records all regions encountered along the best path to s.

$$V(s) = \begin{cases} V_{init}, & if\ s \in \boldsymbol{s_T} \\ -\infty, & if\ s \in \boldsymbol{s_U} \\ max_{s' \in A(s)}[C(s,s') + V(s')], & otherwise \end{cases} \tag{8}$$

$$C(s,s') = \begin{cases} -c_B, & if\ s \in \boldsymbol{s_O} \wedge E(s) > T(s) \\ -2 \cdot c_B, & if\ s \in \boldsymbol{s_O} \wedge E(s) \leq T(s) \\ -c_E, & otherwise \end{cases} \tag{9}$$

$$A(s) = \{s' \mid s' = s + a,\ a \in \{(1,\ 0), (-1,\ 0), (0,\ 1), (0, -1)\}, s' \in \boldsymbol{S}\} \tag{10}$$

$$T(s') = T(s) + 1, s = \underset{s \in A(s')}{\text{argmax}}[C(s,s') + V(s')] \tag{11}$$

$$E(s') = \begin{cases} E(s) + |G_i|, & s \in G_i, G_i \notin L(s) \\ E(s), otherwise \end{cases},\ s = \underset{s \in A(s')}{argmax}[C(s,s') + V(s')] \tag{12}$$

$$L(s') = \begin{cases} L(s) \cup \{G_i\}, & G_i \notin L(s) \\ L(s), & otherwise \end{cases}, s = \underset{s \in A(s')}{argmax}[C(s,s') + V(s')] \tag{13}$$

The remaining buffer space is computed as the number of accessible empty cells minus the path length. The path length $T(s)$ consists of two components: $T_E(s)$, the number of empty cells traversed, and $T_B(s)$, the number of obstructing blocks traversed. Similarly, the accessible empty space $E(s)$ can be divided into $T_E(s)$, the empty cells along the path, and $E_F(s)$, the empty cells outside the path but connected to it. Hence, $E(s) - T(s) = \big(T_E(s) + E_F(s)\big) - \big(T_E(s) + T_B(s)\big) = E_F(s) - T_B(s)$, which corresponds to the available buffer space, defined as the number of connected empty cells outside the path minus the number of obstructing blocks encountered. The procedure for updating cell values is presented in Algorithm 4. As in the MBP, Algorithm 3 is then applied to derive the retrieval path by following the cells with the highest values from the entrance.

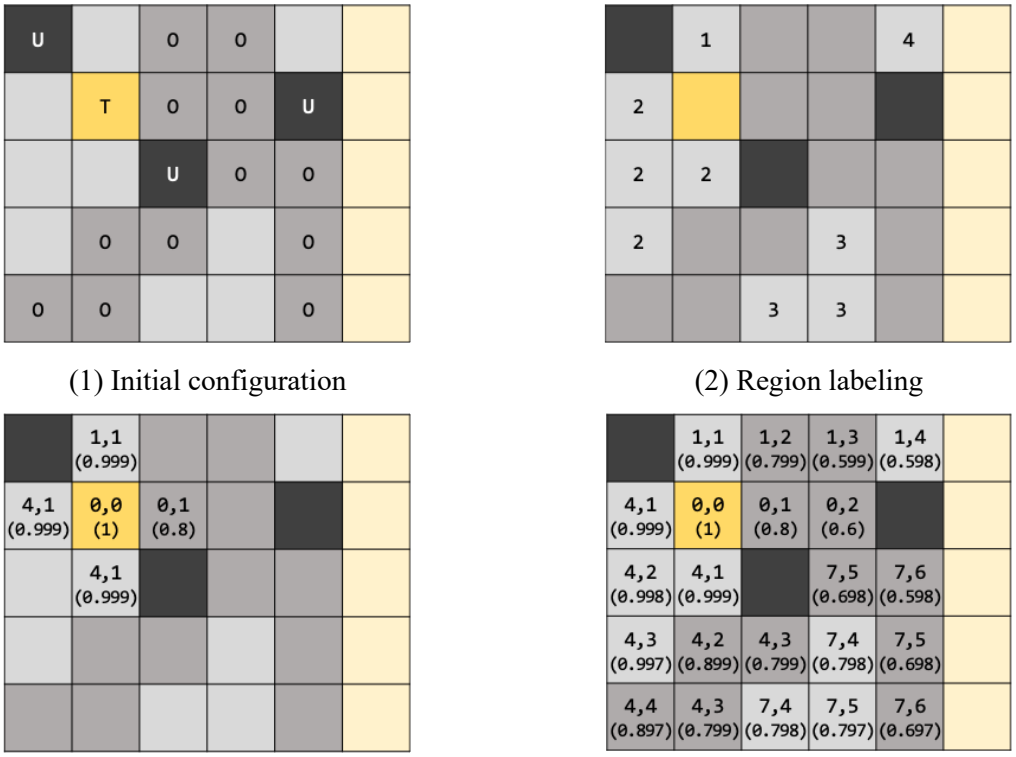
Algorithm 4 Value propagation Ⅱ

Input: $\boldsymbol{S}, \boldsymbol{G}, s_T$
Output: $\boldsymbol{V}$
Initialize variables:
set $V(s_T) \leftarrow V_{init}$
set $E(s_T) \leftarrow 0$
set $T(s_T) \leftarrow 0$
Initialize queue $\boldsymbol{Q}$ as empty
Adjacent directions, $A \leftarrow \{(0,1),(0,-1),(1,0),(-1,0)\}$

push s_T into $\boldsymbol{Q}$
while $\boldsymbol{Q}$ is not empty **do**
  Pop s from Q
  for each a ∈ A **do**
    $s' \leftarrow s + a$
    if $s' \in (\boldsymbol{S_E} \cup \boldsymbol{S_B})$ and $V(s') \leq V(s) + C(s,s')$ **then**
      if $s' \in \boldsymbol{S_E}$ and $\exists G_i \ni s',\ G_i \notin L(s)$ **then**
        $E(s') \leftarrow E(s) + |G_i|$
        $L(s') \leftarrow L(s) \cup \{G_i\}$
      end if
      $T(s') \leftarrow T(s) + 1$
      $V(s') \leftarrow V(s) + C(s,s')$
      Push s' into $\boldsymbol{Q}$
    end if
  end for
end while
return $\boldsymbol{V}$

Algorithm 4. Value propagation for buffer-aware path algorithm

Figure 6 presents an illustrative example of the BAP algorithm. For the problem instance shown in panel (1), the initial value at the target V_{init} is set to 1, the cost of traversing an obstructing block c_B to 0.1, and the cost of traversing an empty cell c_E to 0.001. The figure illustrates the resulting cell-value propagation under the BAP algorithm.



(1) Initial configuration (2) Region labeling

(3) Buffer-aware value propagation (4) Propagated value map

Figure 6. Application of the buffer-aware path algorithm

The following section addresses placement decision algorithms designed to reduce cumulative relocations, building on the simulation framework introduced in Section 3.2

5. Placement algorithms

This section first introduces the placement rules applied in this study and then presents two approaches: the Simulated Greedy Placement (SGP) algorithm, based on a greedy strategy, and a deep reinforcement learning algorithm integrated with graph neural networks.

5.1. Placement rules

Placement decisions are classified into three cases: placing newly arriving blocks, relocating blocks within the yard, and restoring temporarily removed blocks.

For newly arriving blocks, candidate placement positions are restricted to empty cells that can be reached from the entrance with the minimum number of relocations. Priority is given to locations directly accessible without relocation, and if no such location exists, positions requiring the fewest relocations are considered. Let the retrieval algorithm be denoted by φ, where $P_{\varphi}(\boldsymbol{S}, s_T, W)$ represents the retrieval path returned or yard state $\boldsymbol{S}$, target block position s_T, and transporter capacity W. The set of candidate positions is then defined as Eq. (14)

$$\boldsymbol{S_P} = \left\{ s \in \boldsymbol{S_E} \mid R\left(\boldsymbol{S}, P_\varphi(\boldsymbol{S}, s, W)\right) = \min_{s' \in \boldsymbol{S_E}} R\left(\boldsymbol{S}, P_\varphi(\boldsymbol{S}, s', W)\right) \right\} \tag{14}$$

Since both placement and retrieval require secured access from the entrance, the number of relocations necessary for placement can be evaluated by assuming a block is placed in an empty cell and computing the corresponding retrieval path.

For restoring blocks that have been temporarily removed, sufficient feasible space is typically created along the path during target retrieval. In such cases, candidate positions are limited to cells accessible from the entrance. However, when multiple blocks remain outside the yard awaiting re-storage, placements that reduce the number of feasible positions available to these blocks are excluded. Similarly, when relocating blocks within the yard, candidate placements that decrease the number of accessible spaces below the number of remaining obstructing blocks are excluded.

These rules provide a greedy restriction on placement candidates to reduce relocations during placement, while also ensuring computational tractability and efficient exploration in this study.

5.2. Simulated Greedy Placement

The Simulated Greedy Placement (SGP) algorithm determines the optimal placement position based on the current state of the yard using a greedy strategy. For each candidate position $s_i \in \boldsymbol{S_P}$, the incoming block is virtually placed, and the total number of relocations required for all blocks with earlier retrieval deadlines is computed. The block is then placed in the position that results in the fewest relocations.

Let b_k denote a block with weight w_k, $\boldsymbol{S_D}$ the set of blocks with earlier retrieval deadlines than $\boldsymbol{b_k}$, $\boldsymbol{W_T}$ the set of maximum load capacities of transporter types, and $F(\boldsymbol{S}, b_k, s_i)$ the transition state when block b_k is placed at position s_i in yard state $\boldsymbol{S}$. The placement decision under the SGP algorithm is then given by Eq. (15).

$$s = \underset{s_i \in \boldsymbol{S_P}}{\operatorname{argmin}} \left(\sum_{s_j \in \boldsymbol{S_D}} \sum_{\substack{W \in \boldsymbol{W_T} \\ W > w_k}} R\left(\boldsymbol{S'}, P_\varphi\left(\boldsymbol{S'}, s_j, W\right)\right) \right), \; where \; \boldsymbol{S'} = F(\boldsymbol{S}, b_k, s_i) \tag{15}$$

Because the SGP evaluates all candidate positions and computes relocation counts for all blocks with earlier retrieval deadlines, the algorithm exhibits high computational complexity.

5.3. Graph Reinforcement Learning for Placement

Reinforcement learning is a learning paradigm inspired by animal learning mechanisms, in which an optimal policy is acquired by balancing exploration and exploitation. In this study, reinforcement learning combined with graph representations is employed to learn effective placement strategies. The Proximal Policy Optimization (PPO) algorithm, a policy-gradient method widely recognized as effective for combinatorial optimization, is adopted. The proposed model is structured around states, actions, and rewards, which are described in detail in the following section, where states represent yard conditions, actions denote placement choices, and rewards reflect relocation costs. The overall framework is illustrated in Figure 7.

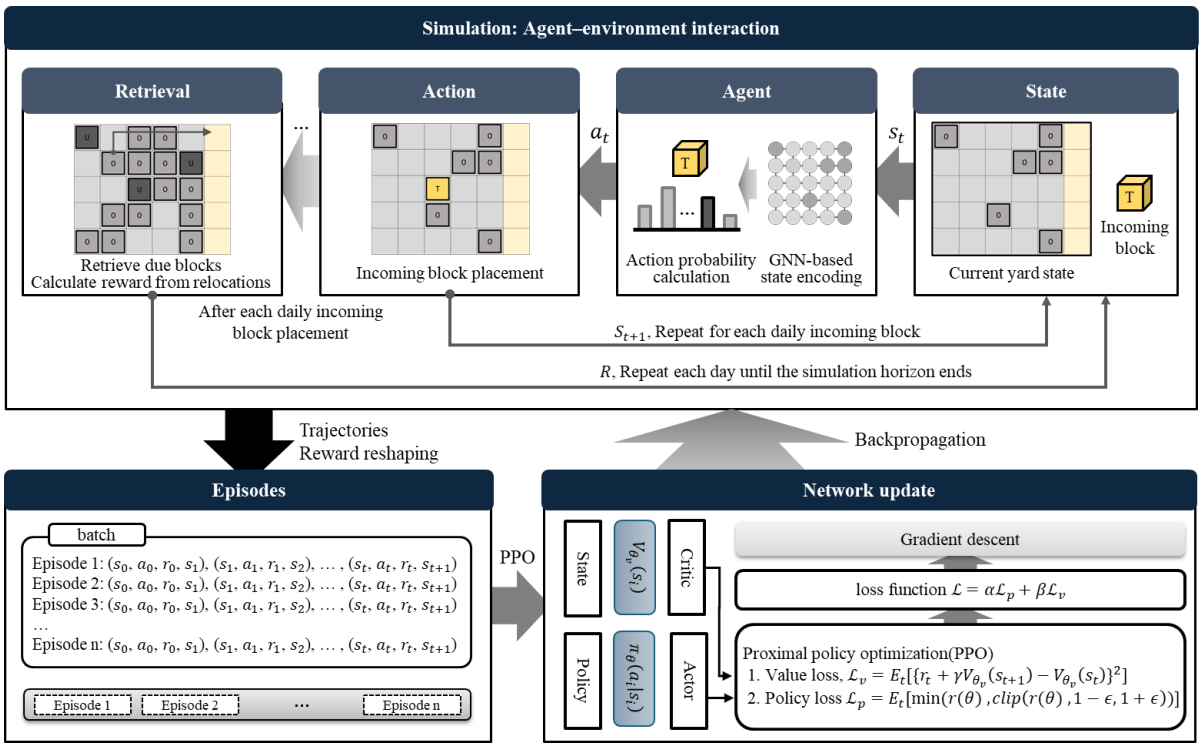


Figure 7. Graph reinforcement learning framework for placement decisions

5.3.1. State

The state represents the environmental information perceived by the agent. In the placement decision problem, the state consists of the current yard configuration together with the block to be placed. The yard is modeled as a two-dimensional grid environment, where each cell is connected to its adjacent cells (up, down, left, and right), and this grid $\boldsymbol{S}$ can be represented as a graph $\boldsymbol{G_s} = (\boldsymbol{V}, \boldsymbol{E})$. Each node $i \in \boldsymbol{V}$ corresponds to a cell s_i, and an edge $(i, j) \in E$ exists if the corresponding cells are adjacent.

Each cell is characterized by the due date d_k and weight w_k of the block b_k stored in it, while an

empty cell is represented with zero values. The feature vector of node i is defined as Eq. (16).

$$v_i = \begin{cases} (d_k, w_k), & \text{if } b_k \text{ is placed in } s_i \\ (0, 0), & \text{if } s_i \text{ is emtpy} \end{cases} \tag{16}$$

For efficient processing, the weight w_k is converted into a binary eligibility vector according to the maximum load capacities of transporter types. State transitions occur at each decision step when a new action is taken. Since placement decisions arise during both block arrivals and relocations of obstructing blocks, the state is updated at each corresponding decision point, as described in Section 5.1.

5.3.2. Graph network encoding

Graph Neural Networks (GNNs) were employed to convey graph-structured data to the agent. In the planar storage problem, the structured grid graph reflects situational characteristics that depend on the states of neighboring cells rather than inherent structural patterns, making spatial GNNs particularly suitable for capturing contextual dependencies.

This study proposes a Cardinal Graph Network (CGN), adapted from the Directional Graph Network (DGN) of Beaini et al. (2021), to account for the directional characteristics of the grid environment. The framework builds on representative spatial GNNs, namely Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT). While conventional spatial GNNs are designed for rotation-invariant graphs, the two-dimensional grid considered here is non-rotation-invariant, with a clearly defined exit direction. This motivates the development of a GNN architecture that explicitly incorporates spatial directionality.

To achieve this, distinct weights are assigned to neighboring nodes in the upward, downward, left, and right directions, in contrast to conventional approaches that apply uniform weights. Through this directional weighting scheme, the Cardinal Graph Convolutional Network (CGCN) is implemented for the GCN framework and the Cardinal Graph Attention Network (CGAT) for the GAT framework.

The convolutional formula of the GCN is given in Eq. (17). Here, H denotes the node representation, with the initial input H^0 corresponding to the feature vector v. W is the trainable weight matrix, $\tilde{A}$ the adjacency matrix including self-loops, $\widetilde{D}$ the degree matrix of $\tilde{A}$ for normalization, and σ the activation function. The superscript l denotes the layer index. In the GCN, information from all neighboring nodes is aggregated with equal weights by applying the same kernel across neighbors.

$$H^{(l+1)} = \sigma(\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}) \tag{17}$$

The Cardinal Graph Convolutional Network (CGCN) extends this formulation to incorporate directional spatial information of the grid. Instead of applying identical weights to all neighbors, distinct weights are assigned to nodes connected in the upward, downward, leftward, and rightward directions, while self-connections are treated separately. Since the grid graph has a fixed number of connections, normalization is performed by dividing by a constant N. The convolutional formula of the CGCN is given in Eq. (18), where U, D, L, R denote the four directions and O denotes the self-connection.

$$H^{(l+1)} = \sigma\left(\frac{\sum_{i\in\{U,D,L,R\}} H_i^{(l)} W_i^{(l)} + H_O^{(l)} W_O^{(l)}}{N}\right) \tag{18}$$

Unlike the GCN, which aggregates information from all neighboring nodes with equal importance, the Graph Attention Network (GAT) performs convolution by assigning different levels of importance to neighbors through an attention mechanism. The convolutional formulation of the GAT is given in Eqs. (19) and (20). Here, H denotes the node representation, W and a are learnable weights, || denotes vector concatenation, and $\mathcal{N}(i)$ is the set of neighboring nodes of node i.

$$e_{ij} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}H_i^{(l)} || \, W^{(l)}H_j^{(l)}]) \tag{19}$$

$$H_i^{(l+1)} = \sigma\left(\sum\nolimits_{j\in\mathcal{N}(i)} \alpha_{ij} H_j^{(l)}\right), \;\; \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\mathcal{N}(i)} \exp(e_{ik})} \tag{20}$$

The Cardinal Graph Attention Network (CGAT) extends this formulation by introducing directional kernels, similar to the CGCN. Specifically, different weight matrices are assigned to neighbors depending on their relative direction. The convolutional formulation of the CGAT is given in Eq. (21), where C denotes the directional notation.

$$e_{ij} = \text{LeakyReLU}\left(a_C^{(l)^T}\left[W_C^{(l)}H_i^{(l)} || W_C^{(l)}H_j^{(l)}\right]\right), C \in \{U, D, L, R\} \text{ (direction from } i \text{ to } j) \tag{21}$$

Spatial GNNs progressively integrate information from a broader neighborhood through multiple layers, and the final node representations are obtained from the last layer of the network.

5.3.3. Action

The action represents the decision made by the agent based on the observed state, corresponding to the selection of a placement location. Since policy-gradient reinforcement learning is employed, the policy is expressed as a probability distribution over the set of feasible candidate locations defined in Section 5.1, and actions are sampled probabilistically from this distribution.

For each node $s_i \in \boldsymbol{S_P}$, the final layer embedding obtained from the graph network is concatenated with the feature vector of the incoming block b_I, and the result is processed by the actor network, implemented as a multi-layer perceptron (MLP). This produces the value α_i of candidate cell s_i, computed as Eq. (22).

$$\alpha_i = \mathrm{MLP}_A(O_i^A), \;\; O_i^A = (H_i^{(l)}, d_I, w_I) \tag{22}$$

where $H_i^{(l)}$ is the final-layer embedding of node i, and (d_I, w_I) denote the due date and weight of block b_I. The policy distribution π is then obtained by applying the softmax function to normalize these values, as shown in Eq. (23).

$$\pi(a_i) = \frac{\exp(\alpha_i)}{\sum_{s_j \in \boldsymbol{S_P}} \exp(\alpha_j)}, \;\; a_i \text{ denotes placing a block } I \text{ into } s_i, s_i \in \boldsymbol{S_P} \tag{23}$$

5.3.4. Reward

The reward represents the evaluation of an action's outcome and is directly aligned with the objective function. Since the goal of this study is to minimize the cumulative number of relocations of obstructing blocks during simulation, the reward is defined as the negative of the relocation count. However, most relocations occur during the retrieval process, which takes place after placement operations have been completed. As a result, rewards are typically assigned only after all placement actions of a given day have been executed. Furthermore, these relocations are often weakly correlated with the most recent placement decision, but instead reflect the cumulative effect of multiple preceding actions.

This creates a structure analogous to an episodic reinforcement learning setting with trajectory-level feedback. Such delayed and sparse rewards give rise to the credit assignment problem, one of the major challenges in reinforcement learning, as it becomes difficult to attribute outcomes to individual actions, thereby hindering effective learning. To address this issue, the study adopts reward redistribution techniques proposed for sparse and delayed rewards(Arjona-Medina et al., 2019; Patil et al., 2020; Ren et al., 2021). In this approach, proxy rewards are reassigned to individual actions after the episode

concludes.

In the placement decision problem, rewards are proportional to the number of relocations and thus take negative values. Instead of assigning relocation-related rewards at the time of retrieval, they are decomposed and redistributed to the earlier actions that placed the obstructing blocks. Specifically, a fixed negative proxy reward $r_c < 0$ is assigned to the placement action responsible for each relocated block. This ensures that the proxy reward remains consistent with the objective of minimizing relocations and is directly used in policy updates, thereby guiding the agent toward learning policies aligned with this goal.

5.3.5. Policy gradient

For policy improvement, the policy gradient method is adopted. The objective is to maximize the expected return of an entire episode, expressed as $J(\theta) = E_{\tau\sim\pi_\theta}[\sum_{t=0}^{T}\gamma^t r_t]$. When $G_k = \sum_{t=k}^{T}\gamma^t r_t$, the gradient of the objective can be approximated as $\nabla_\theta J(\theta) \approx E_{\tau\sim\pi_\theta}[\sum_{t=0}^{T}(\nabla_\theta log\ \pi_\theta G_t)]$.

Unlike conventional policy gradient methods, PPO introduces Generalized Advantage Estimation (GAE) and policy clipping to achieve more stable and effective policy updates. The PPO loss function consists of two terms: the clipped surrogate objective L^{CLIP} for policy improvement and L^{VF} for state value estimation. The clipped surrogate loss is defined as Eq. (24).

$$L^{CLIP}(\theta) = E_t[min(r_t(\theta)\hat{A}_t, clip\ (r_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t)] \tag{24}$$

Where the advantage estimate $\hat{A}_t$ is given by Eq. (25).

$$\hat{A}_t = \sum_{l=0}^{t}(\gamma\lambda)^l\delta_{t+l}\ where\ \ \delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \tag{25}$$

Here, $V_\theta(s)$ denotes the state value, representing the expected return given the current state. As shown in Eq. (26), the critic network is implemented as a multilayer perceptron (MLP). Its input vector O_t^C concatenates the final-layer embeddings of all nodes in the yard graph H_{S_t}, the weight and due date of the incoming block (w_I, d_I), and the number of remaining blocks n_t.

$$V_\theta(s_t) = MLP_C(\ O_t^C\ ), where\ O_t^C = (H_{S_t}, w_I, d_I, n_t) \tag{26}$$

The value function loss for improving state value estimation is defined as Eq. (27).

$$L^{VF}(\theta) = \frac{1}{2} E_t \left[\left(V_\theta(s_t) - V_t^{target} \right)^2 \right], where\ V_t^{target} = r_t + \gamma V_\theta(s_{t+1}) \quad (27)$$

A summary of the graph reinforcement leaning network architecture is presented in Figure 8.

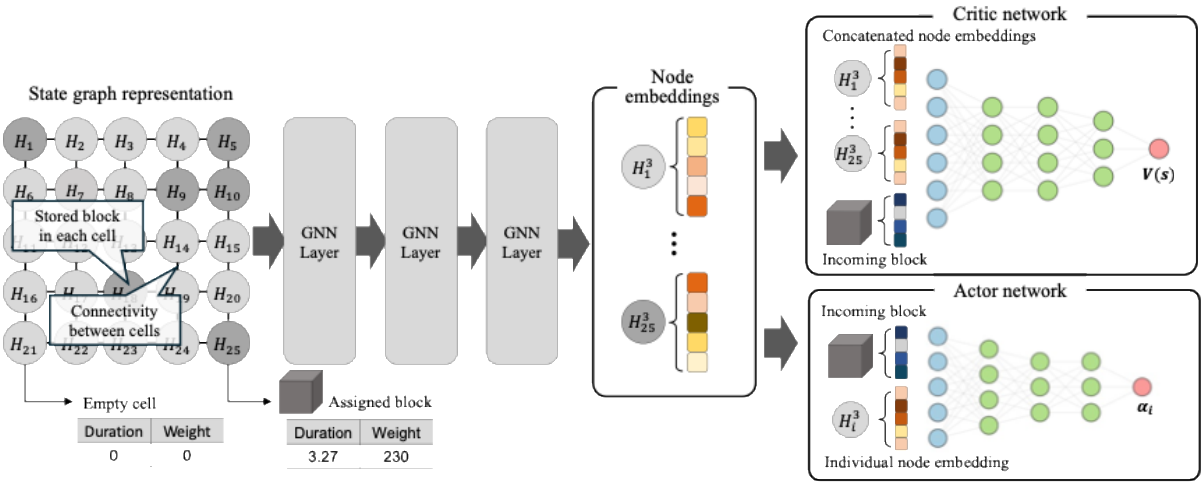


Figure 8. Network structure of graph reinforcement learning for placement algorithm

6. Experiment

This section evaluates the performance of the proposed algorithms through numerical experiments. The problem configuration is summarized in Table 2 and Table 3. Experiments were conducted by varying both the yard size and the distribution of block storage durations. Table 2 presents the yard sizes considered, where the scaling factor S_F is approximately proportional to the yard area and reflects the increase in the number of blocks stored as yard size grows. Table 3 summarizes the number of incoming blocks and the distribution of storage durations. As studied in Kim et al. (2020), block storage durations follow an exponential distribution with mean $1/\lambda_d$. The daily number of incoming blocks u follows a uniform distribution, and I denotes the number of initially stored blocks.

Table 2. Storage yard configurations

Storage type	Type A	Type B	Type C	Type D	Type E	Type F
Storage size	5 · 5	7 · 7	10 · 10	12 · 12	15 · 15	20 · 20
Scaling factor, C_S	1	2	4	6	9	16

Table 3. Scenario parameters

Scenario	$1/\lambda_d$	u	I
1	3	$U(3,6) \cdot C_S$	$5 \cdot C_S$

2	4	$U(2,5) \cdot C_S$	$7.5 \cdot C_S$
3	5	$U(1.5,4.5) \cdot C_s$	$10 \cdot C_S$

Referring to shipyard data, block weights follow the distribution $U(100,500)$. Three types of transporters are assumed, with maximum load capacities of 300, 400, and 500, respectively. The simulation period is set to 15 days. Since the number of relocations naturally increases with the number of incoming blocks, performance is evaluated using the relocation rate, defined as the ratio of cumulative relocations to the total number of blocks handled. Let T denote the simulation horizon, $\bar{u}$ the average number of incoming blocks per day, $N_b = I + \bar{u} \cdot T$ the total number of blocks, and M_T the cumulative number of relocations. The relocation rate ε is computed as Eq. (28).

$$\varepsilon = \frac{M_T}{I + \bar{u} \cdot T} \cdot 100\% \tag{28}$$

For each storage type and scenario, 10 random problem instances were generated to form the evaluation set. Each instance was simulated 10 times, and average results were reported. Since both retrieval and placement algorithms are required for simulation, evaluations were conducted by fixing one and varying the other. Placement was fixed when assessing retrieval, and retrieval was fixed when assessing placement.

6.1. Evaluation of Retrieval Algorithms

For retrieval path planning, the two proposed algorithms, MBP and BAP, were evaluated alongside a rule-based algorithm that determines retrieval paths by finding the shortest distance to the exit, denoted as Shortest Path (SP). To isolate the effect of retrieval algorithms under different conditions, random placement was employed during evaluation. The results are summarized in Table 4, where each entry reports the average relocation rate ε and values in parentheses indicate the percentage difference relative to the BAP algorithm, which generally yielded the best performance.

In most experimental settings, the proposed BAP algorithm outperformed both the MBP and SP approaches. While differences between BAP and MBP were relatively small in smaller yards, and in some cases MBP even performed better, the performance gap widened substantially as yard size increased. On average across scenarios, BAP reduced the number of relocations by 39.67%, 35.0%, and 29.04% compared with MBP. This indicates that BAP is particularly effective in congested scenarios where storage durations are short and block turnover is frequent.

Figure 9 illustrates changes in the average size and number of regions across different yard sizes for each scenario. To account for scale effects, both metrics are normalized by dividing by the square root

of C_s. As yard size increases, the relative size of each region decreases, whereas the number of regions increases, suggesting that smaller free spaces are more densely distributed within larger yards. In small yards, available paths are limited and buffer spaces are scarce. In contrast, larger and more complex yards provide greater path diversity and denser free spaces, making the BAP algorithm, which effectively accounts for buffer availability, increasingly advantageous.

Table 4. Performance of retrieval algorithms over instances

Scenario	**Scenario 1**			**Scenario 2**			**Scenario 3**		
Algorithm	**SP**	**MBP**	**BAP**	**SP**	**MBP**	**BAP**	**SP**	**MBP**	**BAP**
Type A	99.75 (105.57)	48.83 (0.63)	**48.52** **(0.00)**	90.17 (107.92)	**42.63** **(-1.69)**	43.37 (0.00)	86.25 (126.55)	40.00 (5.06)	**38.07** **(0.00)**
Type B	187.17 (82.25)	108.81 (5.95)	**102.70** **(0.00)**	144.13 (136.67)	66.98 (9.99)	**60.90** **(0.00)**	148.98 (119.62)	73.71 (8.66)	**67.84** **(0.00)**
Type C	245.13 (163.68)	114.20 (22.84)	**92.97** **(0.00)**	193.10 (240.66)	63.92 (12.76)	**56.68** **(0.00)**	229.88 (156.88)	101.29 (13.19)	**89.49** **(0.00)**
Type D	327.03 (164.66)	167.37 (35.45)	**123.57** **(0.00)**	248.46 (254.51)	91.51 (30.57)	**70.08** **(0.00)**	305.21 (142.97)	154.90 (23.31)	**125.60** **(0.00)**
Type E	368.60 (205.15)	187.11 (54.90)	**120.79** **(0.00)**	311.84 (264.33)	121.46 (41.91)	**85.59** **(0.00)**	293.46 (260.44)	109.08 (33.98)	**81.42** **(0.00)**
Type F	477.82 (258.93)	241.98 (81.77)	**133.12** **(0.00)**	410.65 (317.54)	173.69 (76.60)	**98.35** **(0.00)**	420.70 (264.46)	189.29 (63.98)	**115.43** **(0.00)**
Avg	284.25 (174.34)	144.72 (39.67)	**103.61** **(0.00)**	233.06 (236.97)	93.37 (35.00)	**69.16** **(0.00)**	247.42 (186.65)	111.38 (29.04)	**86.31** **(0.00)**

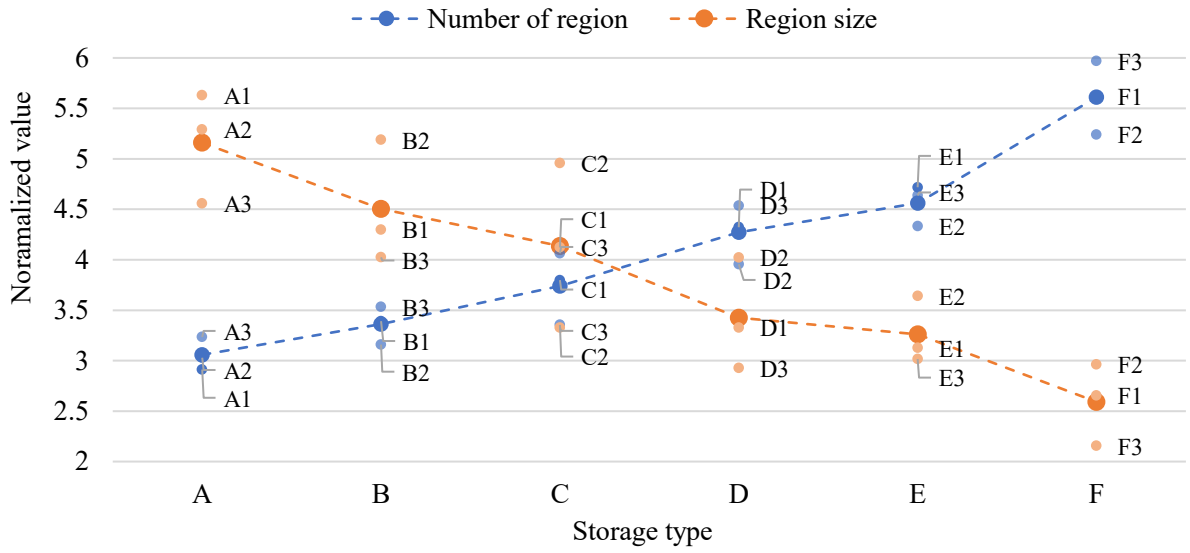


Figure 9. Effect of yard size on region characteristics

6.2. Evaluation of placement algorithms

For placement decision algorithms, the proposed simulation-based greedy algorithm (SGP) and graph reinforcement learning–based algorithms were evaluated alongside widely used placement rules, namely Bottom-Left-Fill (BLF) and Maximum Neighboring Freespace (MNF), the latter designed to reduce path interference by distributing blocks as far apart as possible. To further assess the effectiveness of graph networks, a baseline reinforcement learning algorithm without graph structures, denoted RL-Naïve, was included. Graph-based reinforcement learning algorithms were distinguished according to the type of network employed and labeled as RL-GCN, RL-CGCN, RL-GAT, and RL-CGAT. The evaluation of placement decision algorithms was conducted within simulation environments where the BAP retrieval algorithm, identified as the most effective retrieval approach in the previous section, was applied.

For training, graph reinforcement learning algorithms were trained on Scenario 1 of the medium-sized Type B yard to improve efficiency. In contrast, RL-Naïve, which requires size-specific training, was independently trained for Scenario 1 within each yard type. The main network parameters for the reinforcement learning algorithms are summarized in Table 5.

Table 5. Network structures for the placement algorithms

Depth	Network type	Parameter	Value
1	GNN layer	Number of layers	3
		Hidden dimension	128
		Activation function	ReLU
2	Actor network	Number of layers	3
		Hidden dimension	128,128,64
		Activation function	ReLU
3	Critic network	Number of layers	3
		Hidden dimension	128,128,64
		Activation function	ReLU

The placement algorithms were evaluated separately for small scale yards, Types A, B, and C, and large-scale yards, Types D, E, and F. Due to computational limits, the simulation-based greedy placement, SGP, was tested only on the smaller yards. Figure 10 and Figure 11 present the overall performance of the algorithms for small and large scales, respectively, while Figure 12 and Figure 13 show results for individual instances.

In the small-scale cases, the proposed SGP achieved the best performance, significantly surpassing all other methods. Among reinforcement learning–based approaches, RL-GAT and RL-CGCN yielded

competitive results. On average, SGP achieved a 66.8% improvement over BLF, whereas RL-CGCN and RL-GAT achieved 14.9% and 18.9% fewer relocations than BLF, respectively. BLF occasionally showed comparable results in certain small instances.

For large-scale problems, RL-CGCN demonstrated the strongest performance. RL-GCN and RL-CGAT performed worse than RL-CGCN and RL-GAT but still outperformed rule-based heuristics such as BLF and MNF. RL-CGCN and RL-GAT showed 34.7% and 19.4% improvement over BLF, respectively, and the performance gap between BLF and RL-CGCN widened as the yard size increased, indicating the effectiveness of RL-CGCN in larger problem settings. RL-GAT, however, maintained relatively stable performance across different yard sizes.

Although performance levels fluctuated across different scenarios and yard types, the relative ranking of the algorithms remained broadly consistent, demonstrating that the superiority of the proposed SGP and RL-CGCN is robust under varied problem settings. MNF, despite being a rule-based decision expected to be effective, proved unsuitable under dense yard conditions. Notably, all graph-based reinforcement learning methods outperformed the RL-Naïve baseline in most cases, underscoring the necessity of incorporating graph networks. This also demonstrates that the proposed approach in this study outperforms the method presented by Kim et al. (2020), which is the most closely related prior work. Among them, the proposed CGCN showed clear advantages, as RL-CGCN consistently outperformed other GNN-based reinforcement learning algorithms. However, RL-CGAT failed to improve over RL-GAT, likely because adding directional kernels was redundant given that attention already accounts for neighbor importance.

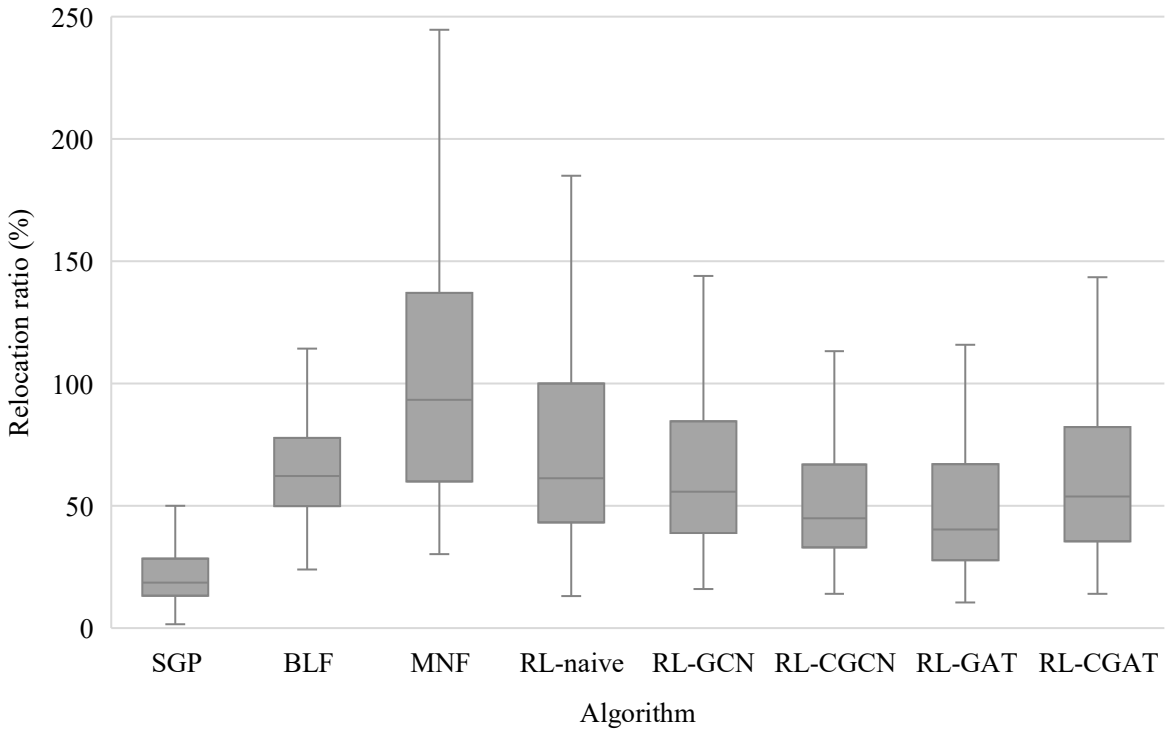


Figure 10. Comparison of placement algorithms in a small storage yard (Types A-C)

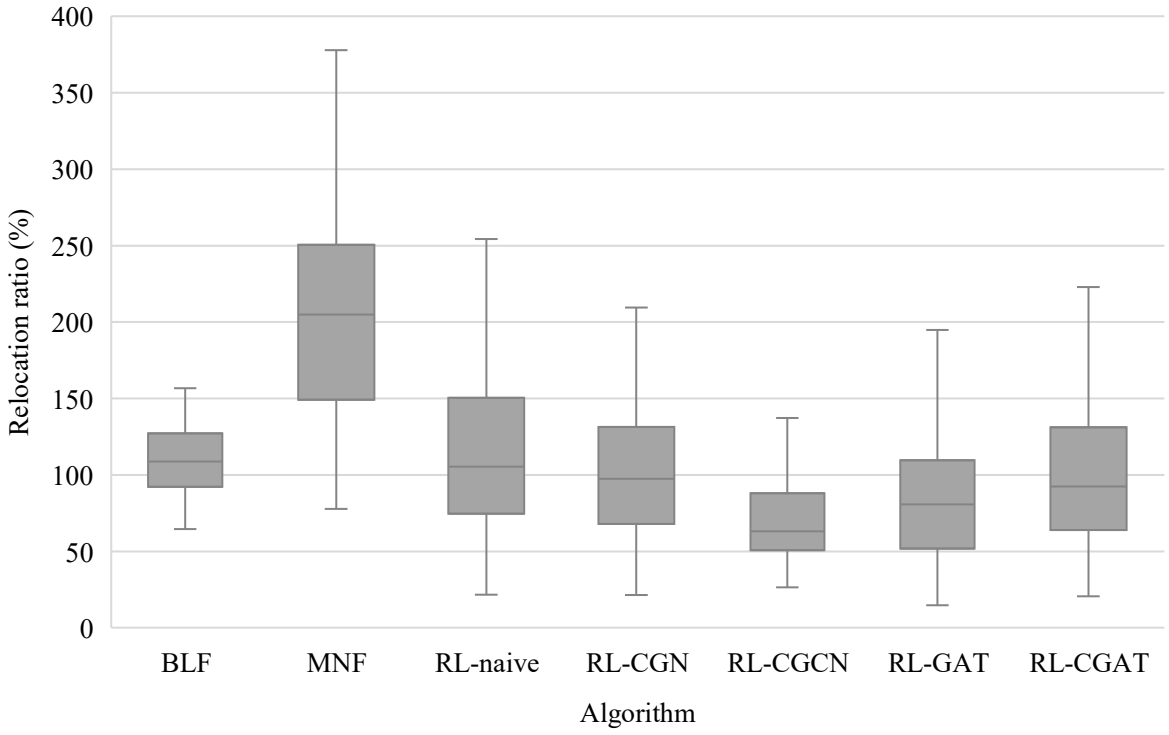


Figure 11. Comparison of placement algorithms in a large storage yard (Types D-F)

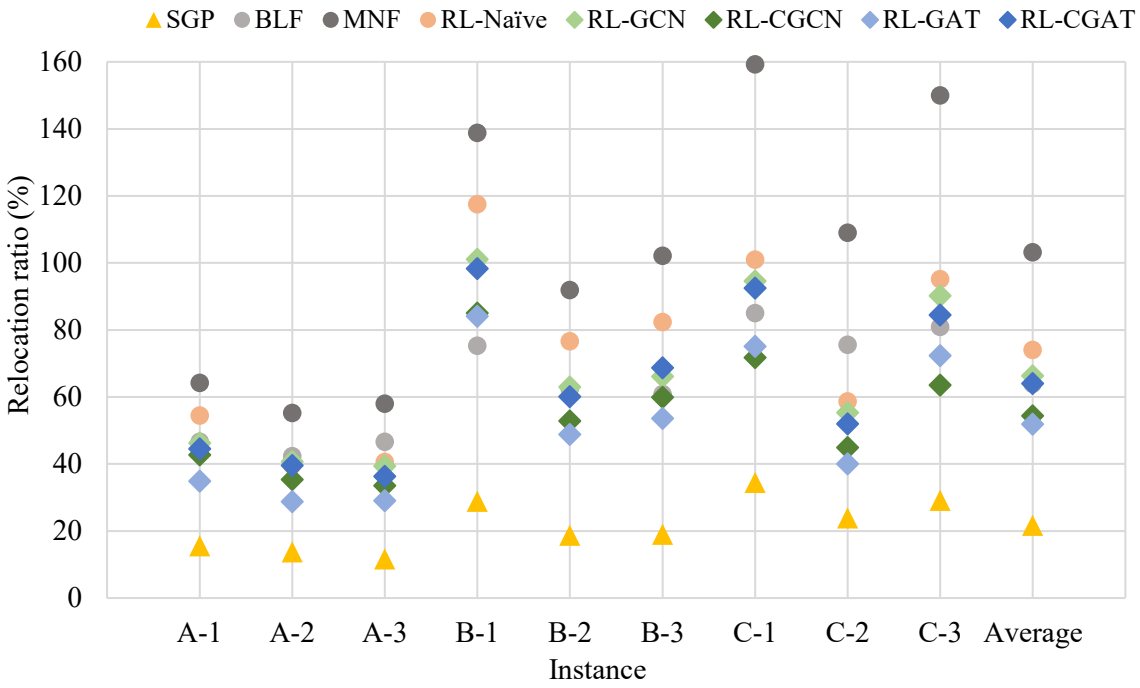


Figure 12. Performance of placement algorithms in small scale instances

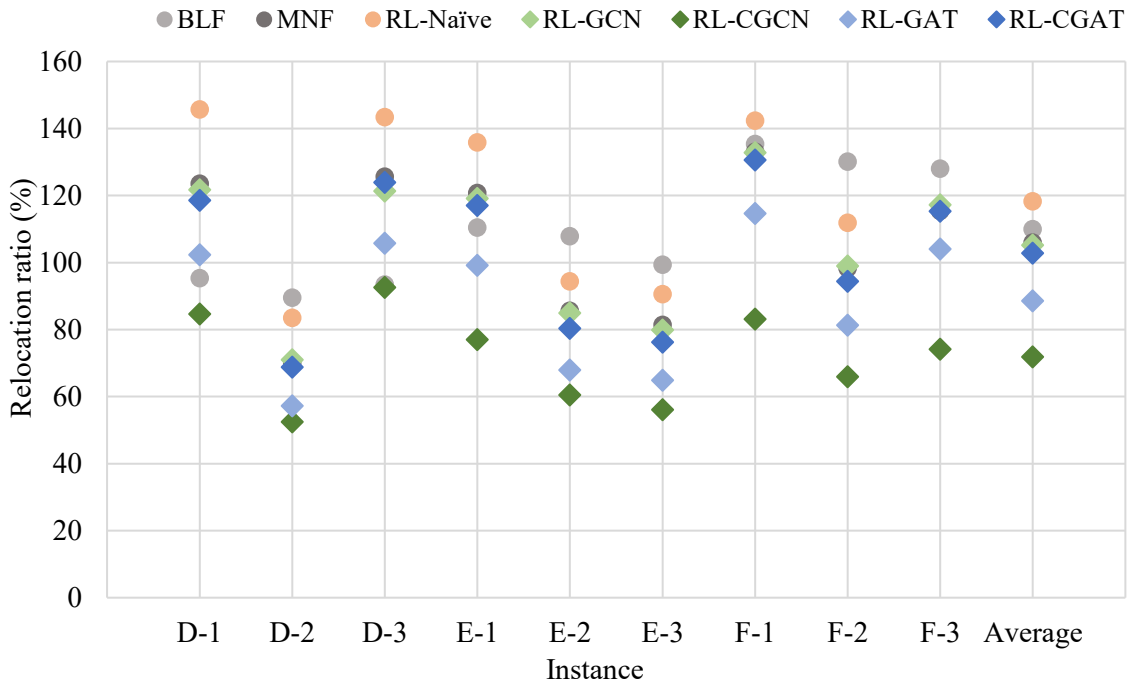


Figure 13. Performance of placement algorithms in large scale instances

Figure 14 presents the computation time of the SGP algorithm as a function of yard size, measured as the maximum decision time within a single simulation. The results indicate that computation time increases approximately in proportion to $O(C_S{}^3)$. For large scale yards, such as Types D, E, and F, the computation required for each placement decision became impractically large, making the algorithm unsuitable for real-time application. In contrast, RL-based algorithms provided decisions within seconds regardless of yard size, allowing for dynamic application. Consequently, SGP was evaluated only on small-scale yards. Moreover, because ten evaluation sets were generated for each environment and each was repeated ten times, the overall computation further constrained large-scale experiments.

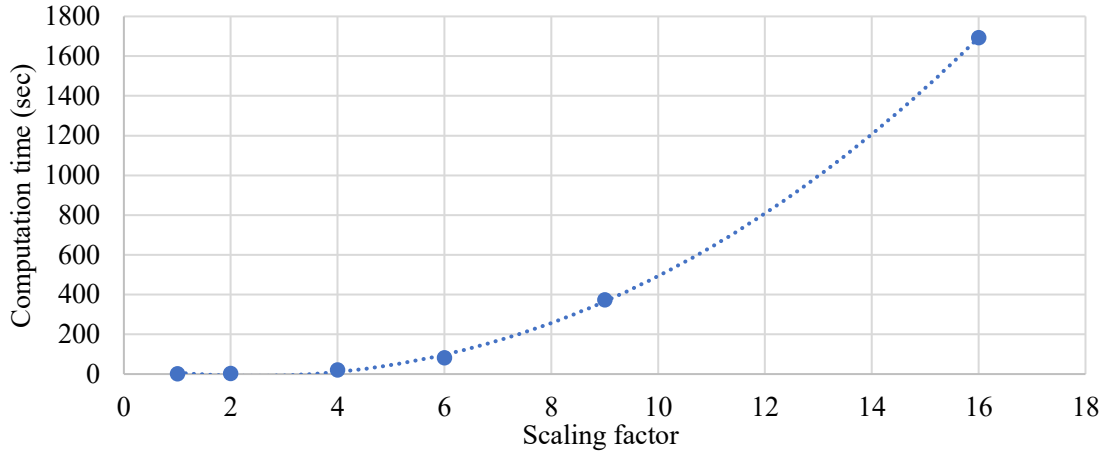


Figure 14. Computation time curve of the SGP algorithm

6.2.1. Placement strategy analysis

In this section, the placement strategies of the three best algorithms in the experiments are examined: SGP, RL-GAT, and RL-CGCN. Figure 15 shows the resulting block configuration for the C-1 instance. The dark brown area on the right denotes the entrance, gray cells indicate placed blocks. Both SGP and RL-GAT adopt a dispersed strategy, spreading blocks widely while maintaining a central corridor to enhance accessibility. In contrast, RL-CGCN concentrates blocks near the center while preserving peripheral corridors. Despite these differences, all three algorithms maintain a small number of large connected empty regions, suggesting a shared tendency to preserve accessibility in the yard.

Figure 16 presents placement states from the perspective of remaining storage duration. Darker blocks indicate shorter remaining durations, while lighter blocks represent longer remaining durations. Across all algorithms, placement strategies consistently enhanced retrieval efficiency by improving accessibility to blocks with imminent retrieval deadlines and positioning them in closer proximity. Notably, RL-CGCN further improved efficiency by placing long-duration blocks deeper inside the yard and positioning short-duration blocks toward the periphery. As yard size increases, maintaining broad

corridors with high accessibility becomes increasingly complex and less effective. In contrast, the compact placement strategy of RL-CGCN, which concentrates long-duration blocks centrally, proved more advantageous under larger problem scales.

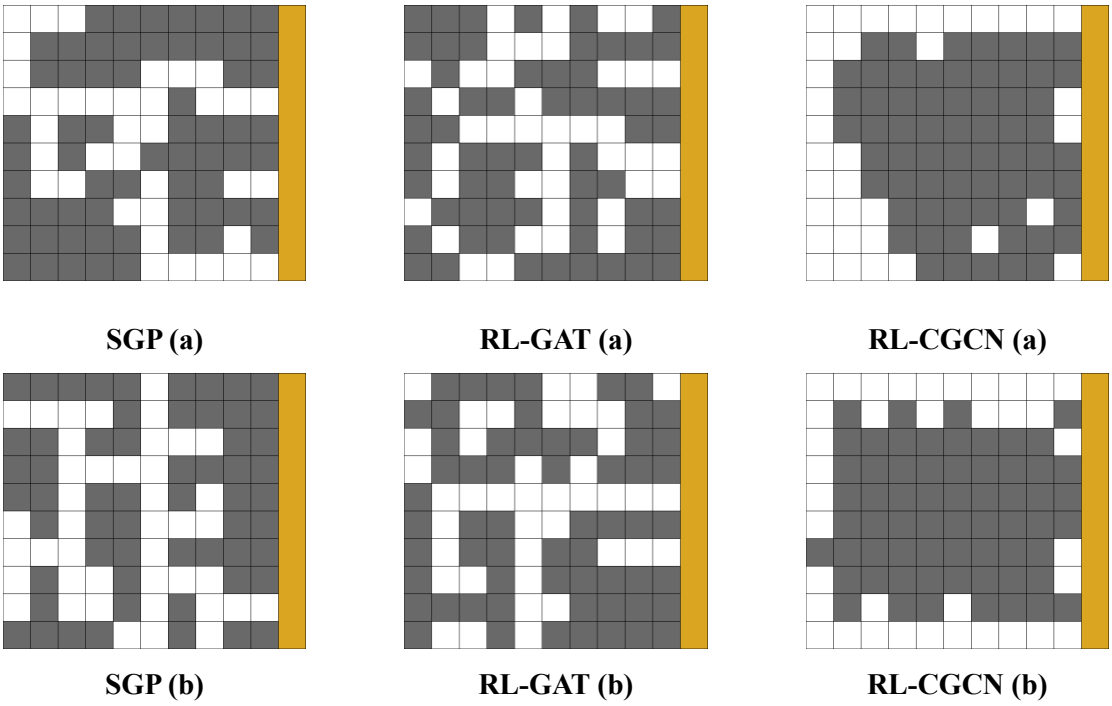


Figure 15. Comparison of placement patterns across SGP, RL-GAT, and RL-CGCN

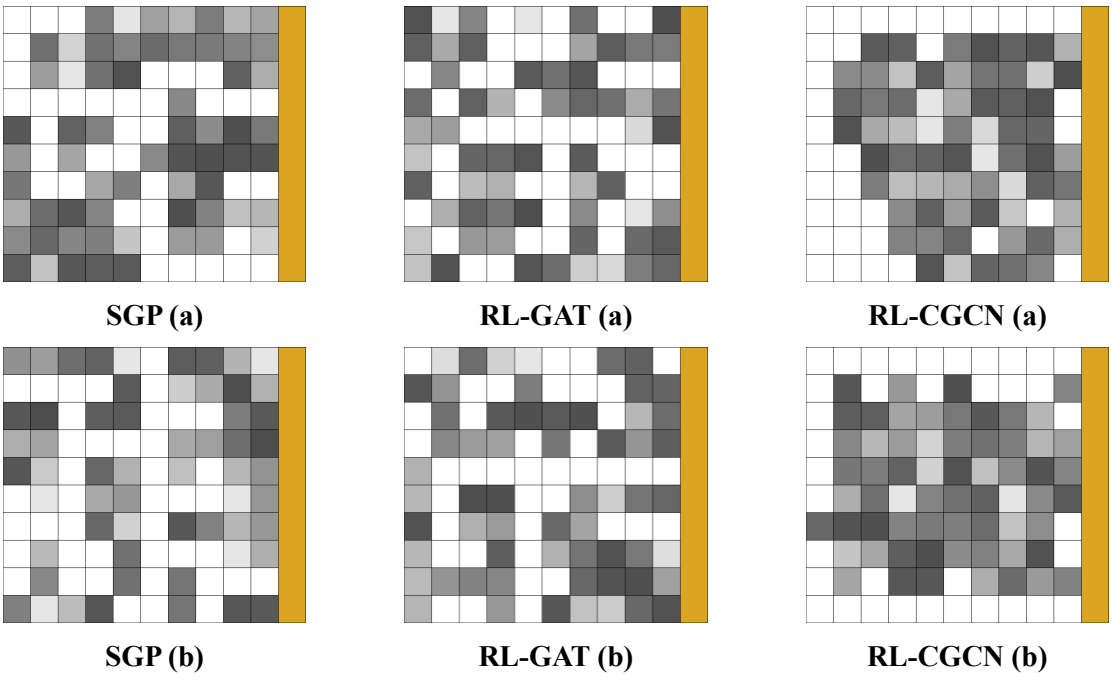


Figure 16. Placement patterns with respect to storage duration

These variations in placement strategies across different graph reinforcement learning models can be interpreted as a result of how each graph network encodes state information. GAT, which determines importance weights based solely on neighboring node features without explicitly modeling geometric orientation, tends to maintain general connectivity corridors without directional preference. In contrast, CGCN, which incorporates geometric directional information, learns strategies that form corridors at the left and right boundaries while densely placing blocks at the center, reflecting the spatial characteristics of the yard.

7. Conclusion

This study addressed retrieval path planning and placement decisions in planar storage yards for shipyard hull blocks, with the objective of minimizing relocations of obstructing blocks. To this end, two dynamic programming-based retrieval algorithms, MBP and BAP, were developed along with two placement approaches: a simulation-based greedy placement algorithm and a reinforcement learning method enhanced with graph neural networks, including a cardinal variant. The experimental results demonstrated that the proposed methods not only consistently outperformed conventional rule-based heuristics but also offered complementary strengths. The retrieval algorithms guaranteed stable pathfinding despite fixed blocks, while the placement algorithms combined solution quality through greedy strategies with efficient dynamic decision-making enabled by reinforcement learning in large-scale yards.

The retrieval algorithms were designed to approximate relocation counts by propagating values backward from the target block. MBP produced simple approximate paths by minimizing the number of obstructing blocks encountered, whereas BAP incorporated buffer space availability to provide a more refined estimate. While BAP generally achieved superior results, it exhibited limitations in fully accounting for buffer space near the entrance. For placement, the greedy algorithm achieved strong performance by simulating all candidate positions, though at the cost of high computational complexity. In contrast, reinforcement learning–based approaches achieved lower performance than the greedy algorithm but enabled fast decision making suitable for large scale and dynamic yard environments. The integration of graph neural networks significantly improved performance over a naïve RL baseline, with the proposed CGNN proving particularly effective by embedding directional information into the graph structure. Interestingly, different GNN architectures led to distinct placement strategies, reflecting differences in state encoding. Furthermore, the use of proxy rewards alleviated the delayed reward problem and enhanced learning efficiency.

This study contributes by incorporating practical constraints such as weight limits and buffer space, which have often been overlooked in prior research. Nonetheless, several aspects remain unaddressed, including storage yards with multiple entrances, transporter access conditions, and placement strategies

that account for actual block geometries and dimensions. Future work should extend the proposed framework to incorporate these elements, thereby advancing toward more realistic and practical algorithms for shipyard logistics.

References

Arjona-Medina, J. A., Gillhofer, M., Widrich, M., Unterthiner, T., Brandstetter, J., & Hochreiter, S. (2019). Rudder: Return decomposition for delayed rewards. *Advances in Neural Information Processing Systems*, *32*.

Beaini, D., Passaro, S., Létourneau, V., Hamilton, W., Corso, G., & Liò, P. (2021). Directional graph networks. International Conference on Machine Learning,

Cinar, Z. M., & Zeeshan, Q. (2020). Design and optimization of automated storage and retrieval systems: A review. Global Joint Conference on Industrial Engineering and Its Application Areas,

Gagliardi, J.-P., Renaud, J., & Ruiz, A. (2012). Models for automated storage and retrieval systems: a literature review. *International journal of production research*, *50*(24), 7110–7125.

Han, S. W., Oh, S.-h., & Woo, J. H. (2025). Crystal Graph Convolutional Neural Network-Based Reinforcement Learning for Adaptive Shipyard Block Transportation Scheduling. *Available at SSRN 5183857*.

Hansen, J. R., Fagerholt, K., Stålhane, M., & Rakke, J. G. (2020). An adaptive large neighborhood search heuristic for the planar storage location assignment problem: application to stowage planning for Roll-on Roll-off ships. *Journal of Heuristics*, *26*, 885–912.

He, P., Zhao, Z., Zhang, Y., & Fan, P. (2024). Optimization of storage and retrieval strategies in warehousing based on enhanced genetic algorithm. *IEEE Access*.

Jeong, Y.-K., Ju, S., Shen, H., Lee, D. K., Shin, J. G., & Ryu, C. (2018). An analysis of shipyard spatial arrangement planning problems and a spatial arrangement algorithm considering free space and unplaced block. *The International Journal of Advanced Manufacturing Technology*, *95*(9), 4307–4325.

Kim, B., Jeong, Y., & Shin, J. G. (2020). Spatial arrangement using deep reinforcement learning to minimise rearrangement in ship block stockyards. *International Journal of Production Research*, *58*(16), 5062–5076.

Kim, B. S., & Joo, C. M. (2012). Ant colony optimisation with random selection for block transportation scheduling with heterogeneous transporters in a shipyard. *International Journal of Production Research*, *50*(24), 7229–7241.

Li, Y., & Li, Z. (2022). Shuttle-based storage and retrieval system: A literature review. *Sustainability*, *14*(21), 14347.

Nam, S.-H., Oh, S.-H., Yoon, H.-C., Cho, Y.-I., Cho, K.-Y., Kwak, D.-H., & Woo, J. H. (2022). Development of des application for factory material flow simulation with simpy. 2022 Winter Simulation Conference (WSC),

Oh, S. H., Cho, Y.-I., Oh, H., Baek, J., Han, S. W., & Woo, J. H. (2024). State Feature Design Framework on Job Shop Scheduling: Graph Attention and Transformer-based Reinforcement Learning. *Authorea Preprints*.

Park, C., & Seo, J. (2009). Mathematical modeling and solving procedure of the planar storage location assignment problem. *Computers & Industrial Engineering*, *57*(3), 1062–1071.

Park, C., & Seo, J. (2010). Comparing heuristic algorithms of the planar storage location assignment problem. *Transportation Research Part E: Logistics and Transportation Review*, *46*(1), 171–185.

Park, C., Seo, J., Kim, J., Lee, S., Baek, T.-H., & Min, S.-K. (2007). Assembly block storage location assignment at a shipyard: a case of Hyundai Heavy Industries. *Production Planning and Control*, *18*(3), 180–189.

Patil, V. P., Hofmarcher, M., Dinu, M.-C., Dorfer, M., Blies, P. M., Brandstetter, J., Arjona-Medina, J. A., & Hochreiter, S. (2020). Align-rudder: Learning from few demonstrations by reward redistribution. *arXiv preprint arXiv:2009.14108*.

Ren, Z., Guo, R., Zhou, Y., & Peng, J. (2021). Learning long-term reward redistribution via randomized return decomposition. *arXiv preprint arXiv:2111.13485*.

Roh, M.-I., & Cha, J.-H. (2011). A block transportation scheduling system considering a minimisation of travel distance without loading of and interference between multiple transporters. *International Journal of Production Research*, *49*(11), 3231–3250.

Silva, A., Coelho, L. C., Darvish, M., & Renaud, J. (2020). Integrating storage location and order picking problems in warehouse planning. *Transportation Research Part E: Logistics and Transportation Review*, *140*, 102003.

Tao, N., Jiang, Z., & Qu, S. (2013). Assembly block location and sequencing for flat transporters in a planar storage yard of shipyards. *International Journal of Production Research*, *51*(14), 4289–4301.

Varghese, R., & Yoon, D. Y. (2005). Dynamic spatial block arrangement scheduling in shipbuilding industry using genetic algorithm. INDIN'05. 2005 3rd IEEE International Conference on Industrial Informatics, 2005.,

Woo, J. H., Song, Y. J., Kang, Y. W., & Shin, J. G. (2010). Development of the decision-making system for the ship block logistics based on the simulation. *Journal of Ship Production and Design*, *26*(04), 290–300.

Yang, P., Miao, L., Xue, Z., & Ye, B. (2015). Variable neighborhood search heuristic for storage location assignment and storage/retrieval scheduling under shared storage in multi-shuttle automated storage/retrieval systems. *Transportation Research Part E: Logistics and Transportation Review*, *79*, 164–177.

Zhong, M., Li, C., Hu, Y., Kang, C., Liu, Z., Guo, J., & Xie, Y. (2025). Integrated dynamic scheduling of shipyard blocks considering adaptive adjustment in stockyard layout. *Engineering Optimization*, 1–32.